UNIT 2

APACHE KAFKA

Distributed Event Streaming Platform

Rahul Inchal

Topics Covered

Introduction to Apache Kafka
Need for Kafka in Modern Data Systems
Kafka vs Traditional Message Brokers: RabbitMQ, AWS SQS
Kafka Architecture
Kafka Cluster and Broker
Topics and Partitions
Kafka Producer and Consumer
Message Serialization: Avro, Protobuf and Schema Registry
Consumer Groups
Single-Node and Multi-Broker Kafka Setup
Basic Kafka Commands and Operations

Table of Contents

Table of Contents 2
1. Introduction to Apache Kafka 3
    1.1 Definition 3
    1.2 The Easiest Way to Understand Kafka 3
    1.3 History and Evolution 4
    1.4 What exactly is an Event? 4
    1.5 The Three Capabilities of Kafka 4
    1.6 The Kafka Ecosystem 5
    1.7 Where Kafka is Used — Real Applications 5
2. Need for Kafka in Modern Data Systems 6
    2.1 How Data Systems Have Changed 6
    2.2 Problem 1: The N × M Integration Problem 6
    2.3 The Kafka Solution: N + M 6
    2.4 Problem 2: Batch Processing is Too Slow 7
    2.5 Problem 3: Traditional Queues Cannot Replay 7
    2.6 Problem 4: Tight Coupling Between Services 7
    2.7 Summary of Kafka's Advantages 7
3. Kafka vs Traditional Message Brokers (RabbitMQ, AWS SQS) 9
    3.1 Two Fundamentally Different Philosophies 9
    3.2 RabbitMQ in Brief 9
    3.3 AWS SQS in Brief 9
    3.4 Full Comparison Table 10
    3.5 A Simple Decision Guide 10
4. Kafka Architecture 12
    4.1 The Complete Architecture 12
    4.2 Every Component Explained 12
    4.3 How Data is Physically Stored 12
    4.4 Replication, Leader and ISR 13
    4.5 ZooKeeper versus KRaft 13
    4.6 Why Kafka is Extremely Fast 14
5. Kafka Cluster and Broker 15
    5.1 What is a Broker? 15
    5.2 What is a Cluster? 15
    5.3 The Controller Broker 15
    5.4 What Happens When a Broker Fails — Step by Step 15
    5.5 Fault Tolerance and Replication Factor 16

5.6 Important Broker Configuration Properties 16
6. Topics and Partitions 18
6.1 Topic 18
6.2 Partition 18
6.3 The Ordering Rule — Explained With an Example 18
6.4 How the Partition is Chosen 19
6.5 Offsets 19
6.6 Retention and Log Compaction 20
6.7 Choosing the Number of Partitions 20
7. Kafka Producer and Consumer 21
7.1 The Kafka Producer 21
7.2 The Kafka Consumer 23
7.3 Message Delivery Semantics 25
8. Message Serialization: Avro, Protobuf and Schema Registry 26
8.1 What is Serialization and Why is it Needed? 26
8.2 Why Plain JSON is a Bad Idea at Scale 26
8.3 Apache Avro 26
8.4 Protocol Buffers (Protobuf) 27
8.5 Comparison of Serialization Formats 28
8.6 Schema Registry 28
9. Consumer Groups 31
9.1 What is a Consumer Group? 31
9.2 The Golden Rules 31
9.3 Group Sizes — Three Cases 31
9.4 Two Groups Reading the Same Topic 31
9.5 Group Coordinator and Group Leader 32
9.6 Rebalancing 32
9.7 Partition Assignment Strategies 33
9.8 Important Consumer Group Settings 33
9.9 Consumer Lag — The Key Metric 33
9.10 Scaling a Consumer Group — Worked Example 34
10. Single-Node and Multi-Broker Kafka Setup 35
10.1 Prerequisites 35
10.2 Single-Node Single-Broker Setup 35
10.3 Single-Node Multi-Broker Setup 36
10.4 Multi-Node (True Distributed) Cluster 37
10.5 KRaft Mode Setup (No ZooKeeper) 37
10.6 Running Kafka With Docker (Quick Alternative) 38

10.7 Common Setup Mistakes 38
11. Basic Kafka Commands and Operations 39
11.1 Topic Commands 39
11.2 Producer Commands 39
11.3 Consumer Commands 39
11.4 Consumer Group Commands 40
11.5 Configuration Commands 40
11.6 Testing and Maintenance Commands 41
11.7 Quick Command Reference 41
12. End-to-End Case Study: A Food Delivery App 42
12.1 The Requirement 42
12.2 The Design 42
12.3 Design Decisions Explained 42
12.4 What Happens When Things Go Wrong 42
13. Summary and Quick Revision 44
13.1 Glossary 44
13.2 Points to Remember 44

1. Introduction to Apache Kafka

1.1 Definition

Apache Kafka is an open-source, distributed event streaming platform used to publish, store, process and subscribe to streams of records in real time.

Let us break this definition into simple pieces, because every word in it is important.

Word in the Definition	What It Actually Means
Open-source	Free to use, source code is public, maintained by the Apache Software Foundation
Distributed	Runs on many servers (called brokers) at the same time, not on one machine
Event / Record	A small piece of data describing "something that happened"
Streaming	Data keeps flowing continuously instead of arriving in one big batch
Publish	Applications write data into Kafka
Subscribe	Applications read data from Kafka
Store	Kafka keeps the data safely on disk for a configured time
Real time	Data becomes available to readers within milliseconds

1.2 The Easiest Way to Understand Kafka

Imagine a very long notebook kept in a library. The notebook has the following rules:

- Anyone can come and write a new line, but only at the end of the notebook. Nothing already written can be changed or erased.
- Every line gets a line number automatically: 0, 1, 2, 3, and so on.
- Many readers can read the notebook at the same time. Each reader keeps a bookmark showing which line they have reached.
- Reading a line does not remove it. Ten different readers can read line number 57 independently.
- The library makes photocopies of the notebook and stores them in other rooms, so if one room catches fire, no data is lost.
- Very old pages are thrown away after a fixed number of days to save space.

That notebook is exactly what Kafka is. In Kafka language: the notebook is a partition, the line number is the offset, the writers are producers, the readers are consumers, the bookmark is a committed offset, the photocopies are replicas, and throwing away old pages is retention.

```
    THE "NOTEBOOK" MODEL OF KAFKA

offset:  0   1   2   3   4   5   6   7
     +----+----+----+----+----+----+----+----+
     | A  | B  | C  | D  | E  | F  | G  | H  | <-- new
     +----+----+----+----+----+----+----+----+  writes
           ^         ^     ^    appended
           |         |     |    here
        Reader-1     Reader-2  Reader-3
       (bookmark 2)  (bookmark 5) (bookmark 7)

* Writers only append at the end.
* Readers move their own bookmark, at their own speed.
* Reading does NOT delete anything.
```

Figure 1.1 — Kafka as an append-only log with independent readers

1.3 History and Evolution

Kafka was created because LinkedIn had a real and painful problem. LinkedIn was generating a huge amount of user-activity data (profile views, searches, clicks, page loads) and had built dozens of separate pipelines to move this data to dozens of separate systems. The pipelines were fragile, slow, and impossible to maintain. The team decided to build one unified, high-throughput pipeline instead.

Year / Version	Milestone
2010	Developed internally at LinkedIn by Jay Kreps, Neha Narkhede and Jun Rao
2011	Open-sourced and donated to the Apache Software Foundation
2012	Graduated from the Apache Incubator to a top-level project
2014	Confluent founded by the original creators to support Kafka commercially
2015–2017	Kafka Connect and Kafka Streams added; Kafka becomes a full platform
2017 (v0.11)	Exactly-once semantics and transactions introduced
2021 (v2.8)	KRaft mode released as preview — Kafka without ZooKeeper
2022 (v3.3)	KRaft declared production-ready
2024 (v4.0)	ZooKeeper support fully removed; KRaft becomes the only mode

Why the name "Kafka"? Jay Kreps named it after the writer Franz Kafka, because the system is, in his words, "a system optimised for writing". It has no technical meaning.

1.4 What Exactly is an Event?

An event (also called a message or a record) is the smallest unit of data in Kafka. It represents a fact — something that has already happened and cannot be undone. This is different from a command ("charge this card"), which is a request that may fail. Events are always written in the past tense: "OrderPlaced", "PaymentReceived", "UserLoggedIn".

```
+-------------------------------------------------+
|          ONE KAFKA EVENT                        |
```

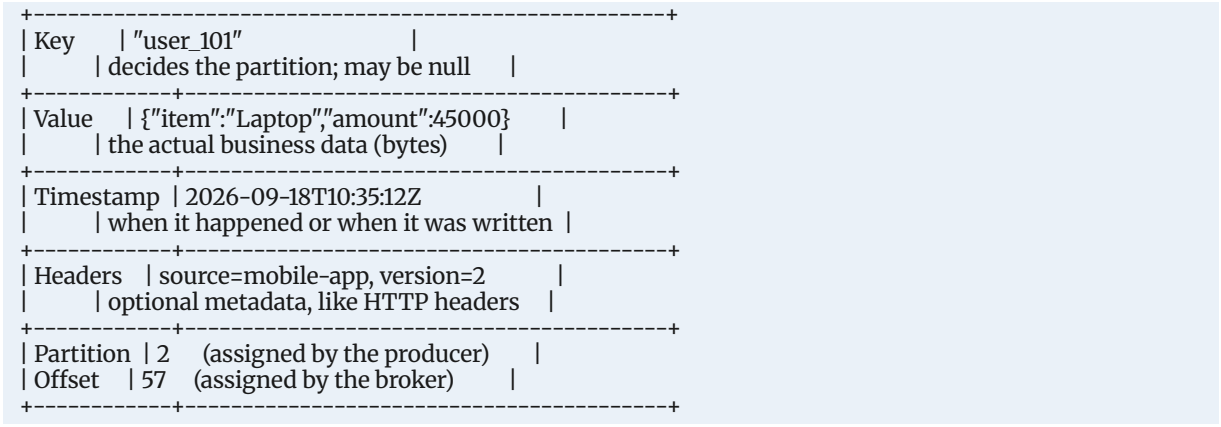


Figure 1.2 — Detailed structure of a Kafka record

Important point about size: Kafka is designed for small messages. The default maximum message size is 1 MB (message.max.bytes). If you need to send a 500 MB video, you store the video in cloud storage such as S3 and send only the URL through Kafka. This is called the claim-check pattern.

1.5 The Three Capabilities of Kafka

Kafka is often confused with a message queue, but it actually combines three different systems into one.

Capability	Traditionally Provided By	What Kafka Gives You	Example
Messaging	RabbitMQ, ActiveMQ, JMS	Publish–subscribe between applications	Order service notifies billing service
Storage	Databases, HDFS	Durable, replicated, ordered log on disk	Keep 7 days of orders and replay them
Stream processing	Spark Streaming, Flink	Filter, join and aggregate data as it flows	Count orders per city every minute

1.6 The Kafka Ecosystem

Around the core broker, Kafka provides four APIs and several supporting tools. You should know their names and purposes.

Component	Purpose
Producer API	Lets an application publish records to topics
Consumer API	Lets an application subscribe to topics and read records
Streams API	A Java library for transforming, joining and aggregating streams
Connect API	Ready-made connectors to move data in and out of Kafka without writing code

Component	Purpose
Admin API	Programmatically create topics, change configs, inspect the cluster
Schema Registry	Stores and versions the schemas of messages (Confluent component)
ksqlDB	Lets you write SQL queries over Kafka streams
MirrorMaker 2	Replicates topics between two different Kafka clusters

1.7 Where Kafka is Used — Real Applications

Use Case	How Kafka Helps	Real Example
Activity tracking	Every click and page view becomes an event	LinkedIn, Flipkart, YouTube recommendations
Log aggregation	Logs from 1000 servers flow into a single stream	Centralised logging into Elasticsearch
Metrics and monitoring	Application metrics streamed to dashboards	Live server-health dashboards
Real-time analytics	Aggregations computed within seconds of an event	Sales in the last 5 minutes by city
Event-driven microservices	Services react to events instead of calling each other	Uber trip lifecycle events
Change Data Capture	Every database insert/update becomes an event	Syncing MySQL to a data warehouse
IoT and telemetry	Millions of sensors publish readings continuously	Smart meters, vehicle GPS tracking
Fraud detection	Each transaction checked against rules in milliseconds	Credit-card fraud alerts
Event sourcing	The event log itself becomes the source of truth	Banking ledgers, audit trails

2. Need for Kafka in Modern Data Systems

2.1 How Data Systems Have Changed

To understand why Kafka became necessary, compare the old world with the new world.

Aspect	Traditional Systems (before ~2010)	Modern Systems (today)
Data volume	Gigabytes per day	Terabytes per day
Data speed	Processed once a night in batches	Must be processed within milliseconds
Architecture	One large monolithic application	Hundreds of small microservices
Data sources	A few databases and files	Web, mobile, IoT, APIs, logs, sensors
Data consumers	One data warehouse	Dashboards, ML models, search, alerts, warehouse
Expectation	Yesterday's report is fine	Live dashboards, instant fraud alerts

2.2 Problem 1: The N × M Integration Problem

In point-to-point integration, every source system is connected directly to every target system. Each connection needs its own code, its own data format, its own error handling and its own monitoring.

With 4 sources and 4 targets you need 16 connections. With 10 and 10 you need 100. With 20 and 20 you need 400. The complexity grows as the product N × M, which is why this model collapses as a company grows.

```
   WITHOUT  KAFKA   (N x M = 16 connections)

Website  ----+----+----+----> Database
         \   |   /
Mobile   ------+--+--+-------> Analytics
         /   |   \
Sensors  ----+----+----+----> Email Service
         \   |   /
Billing  ----+----+----+----> Data Warehouse

Problems:
- Every arrow is separate code that can break
- Adding 1 new target = 4 new connections to build
- Each source must know every target's protocol
- If a target is down, the source is blocked
```

Figure 2.1 — Point-to-point integration becomes unmanageable

2.3 The Kafka Solution: N + M

Kafka sits in the middle as a central hub. Every source writes once to Kafka. Every target reads from Kafka. Now 4 sources plus 4 targets equal only 8 connections. Adding a new target adds exactly one connection, not four.

```
   WITH  KAFKA   (N + M = 8 connections)
```

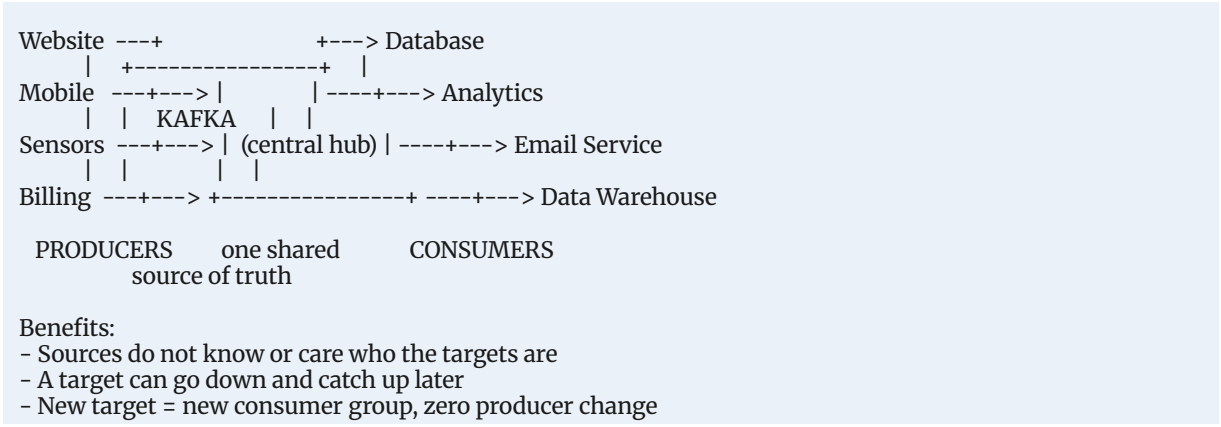


Figure 2.2 — Kafka as the central nervous system of the company

2.4 Problem 2: Batch Processing is Too Slow

In the old ETL model, data was extracted from databases at night, transformed, and loaded into a warehouse by morning. This means business decisions are always based on data that is up to 24 hours old.

Example where this fails: A credit-card company detects fraud using a nightly batch job. A stolen card is used at 10 AM. The fraud is detected at 2 AM the next day — sixteen hours and twenty transactions too late. With Kafka the transaction event reaches the fraud engine within 50 milliseconds and the card is blocked before the second purchase completes.

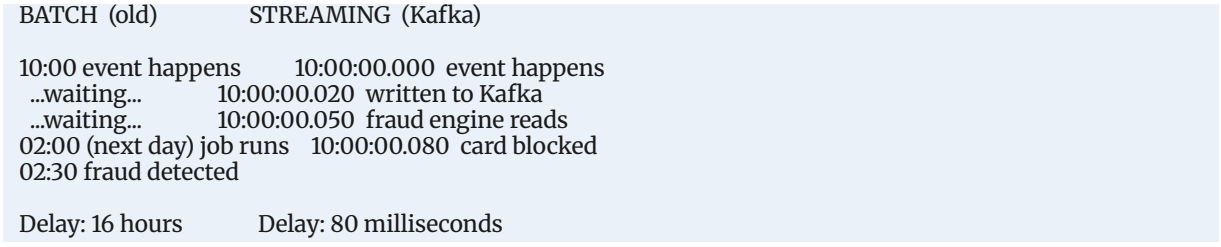


Figure 2.3 — Batch delay versus streaming delay

2.5 Problem 3: Traditional Queues Cannot Replay

In a traditional message queue, a message is deleted as soon as a consumer acknowledges it. That sounds efficient, but it creates a serious operational problem.

Scenario: A consumer has a bug in its calculation logic. It has processed 10 lakh messages incorrectly over three days. With a traditional queue, those messages are gone forever — the data cannot be recovered. With Kafka, the team fixes the bug, resets the consumer group offset back three days, and simply reprocesses everything correctly. This ability to rewind is one of the biggest practical reasons companies choose Kafka.

2.6 Problem 4: Tight Coupling Between Services

When Service A calls Service B directly over HTTP, three things go wrong:

- Availability coupling: if B is down, A fails too.
- Speed coupling: if B is slow, A becomes slow.
- Knowledge coupling: A must know B's address, protocol and data format.

With Kafka, A only writes an event and moves on. If B is down, the event waits safely in Kafka and B processes it when it comes back. This is called temporal decoupling.

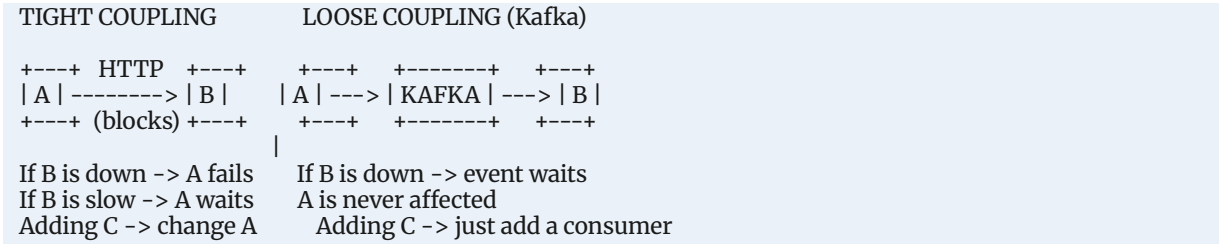


Figure 2.4 — Coupling before and after Kafka

2.7 Summary of Kafka's Advantages

Advantage	Explanation	How Kafka Achieves It
High throughput	Millions of messages per second	Sequential disk I/O, batching, zero-copy, compression
Horizontal scalability	Grow by adding machines, not bigger machines	Partitions spread across many brokers
Durability	Data survives crashes	Written to disk and replicated to other brokers
Fault tolerance	Cluster keeps working when a broker dies	Leader election from the ISR
Decoupling	Producers and consumers are independent	Pull model plus shared topics
Replay	Old data can be read again	Retention based on time, not on consumption
Ordering	Events are read in the order written	Strict ordering inside each partition
Low latency	Typically under 10 milliseconds	In-memory page cache and efficient protocol
Multiple consumers	Same data serves many teams	Independent consumer groups

3. Kafka vs Traditional Message Brokers (RabbitMQ, AWS SQS)

3.1 Two Fundamentally Different Philosophies

Traditional brokers follow "smart broker, dumb consumer". The broker does all the thinking. It tracks the state of every single message, decides which consumer receives it, waits for an acknowledgement, retries if there is no acknowledgement, and finally deletes the message. Because the broker must remember the state of every message, it becomes the bottleneck when the volume grows.

Kafka follows "dumb broker, smart consumer". The Kafka broker does almost nothing intelligent. It appends bytes to a file and serves ranges of bytes when asked. It does not track which consumer read what. Each consumer remembers its own offset. Because the broker is so simple, it can handle enormous volume.

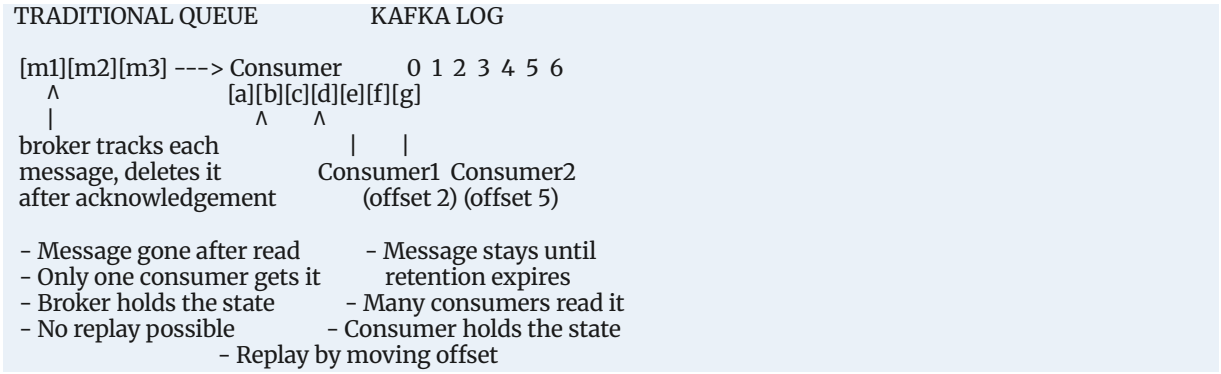


Figure 3.1 — Queue model versus log model

3.2 RabbitMQ in Brief

RabbitMQ is a traditional message broker built on the AMQP protocol. Its strength is flexible routing.

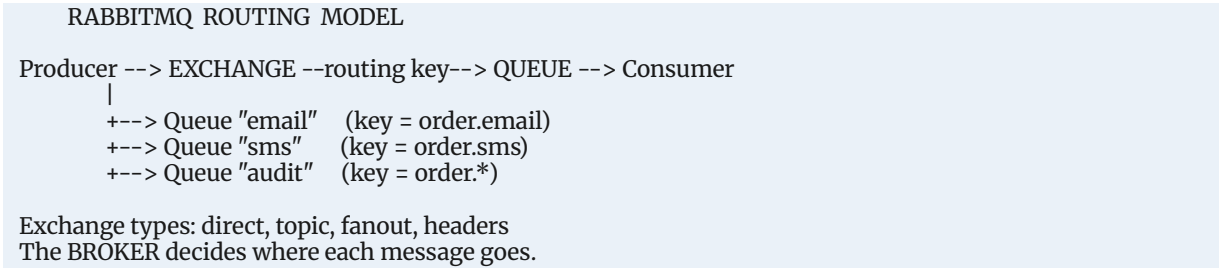


Figure 3.2 — RabbitMQ exchange and queue routing

- Strengths: rich routing rules, message priority, delayed/scheduled delivery, per-message TTL, dead-letter queues, request–reply (RPC) patterns, mature management UI.

- Weaknesses: lower throughput, no replay, clustering is harder, performance drops when queues grow long.

3.3 AWS SQS in Brief

Amazon SQS is a fully managed cloud queue. There is no server to install, patch or scale — AWS handles everything. You simply call an API to send and receive messages.

Queue Type	Throughput	Ordering	Delivery
Standard	Nearly unlimited	Not guaranteed (best effort)	At-least-once (duplicates possible)
FIFO	About 3,000 msg/sec with batching	Strictly guaranteed	Exactly-once processing

- Strengths: zero operations, automatic scaling, pay only for what you use, deep integration with AWS Lambda and SNS.
- Weaknesses: maximum retention is 14 days, message size limit 256 KB, no replay after deletion, AWS lock-in, one queue feeds one consumer group only (SNS is needed for fan-out).

3.4 Full Comparison Table

Feature	Apache Kafka	RabbitMQ	AWS SQS
Category	Distributed event streaming log	Message broker (queue)	Managed cloud queue
Protocol	Custom binary over TCP	AMQP, MQTT, STOMP	HTTPS REST API
Delivery model	Pull (consumer fetches)	Push (broker pushes)	Pull (long polling)
Message retention	Time/size based, default 7 days, kept after reading	Deleted after acknowledgement	Deleted after explicit delete, max 14 days
Replay old data	Yes — reset the offset	No	No
Throughput	Very high, millions per second	Tens of thousands per second	High (Standard), low (FIFO)
Latency	Very low, 2–10 ms	Very low, under 1 ms	Higher, tens of ms (HTTP)
Ordering	Guaranteed per partition	Guaranteed per queue	Only in FIFO queues
Routing logic	Simple: topic and partition	Rich: direct, topic, fanout, headers	Very simple, none
Fan-out to many readers	Native, via consumer groups	Needs a fanout exchange and extra queues	Needs SNS plus several queues
Message size	Default 1 MB, configurable	Up to 128 MB (not advised)	256 KB (2 GB via S3 extension)

Feature	Apache Kafka	RabbitMQ	AWS SQS
Priority messages	Not supported	Supported	Not supported
Delayed delivery	Not built in	Supported via plugin	Supported up to 15 minutes
Scaling	Add brokers and partitions	Add nodes, clustering is complex	Fully automatic by AWS
Storage model	Disk log, replicated	Mainly memory, disk optional	Managed by AWS
Operational effort	High — you run the cluster	Medium	Almost zero
Cost model	Server and storage cost	Server cost	Per request and per GB
Best suited for	Streaming, analytics, event sourcing, very high volume	Task queues, complex routing, RPC	Simple decoupling inside AWS

3.5 A Simple Decision Guide

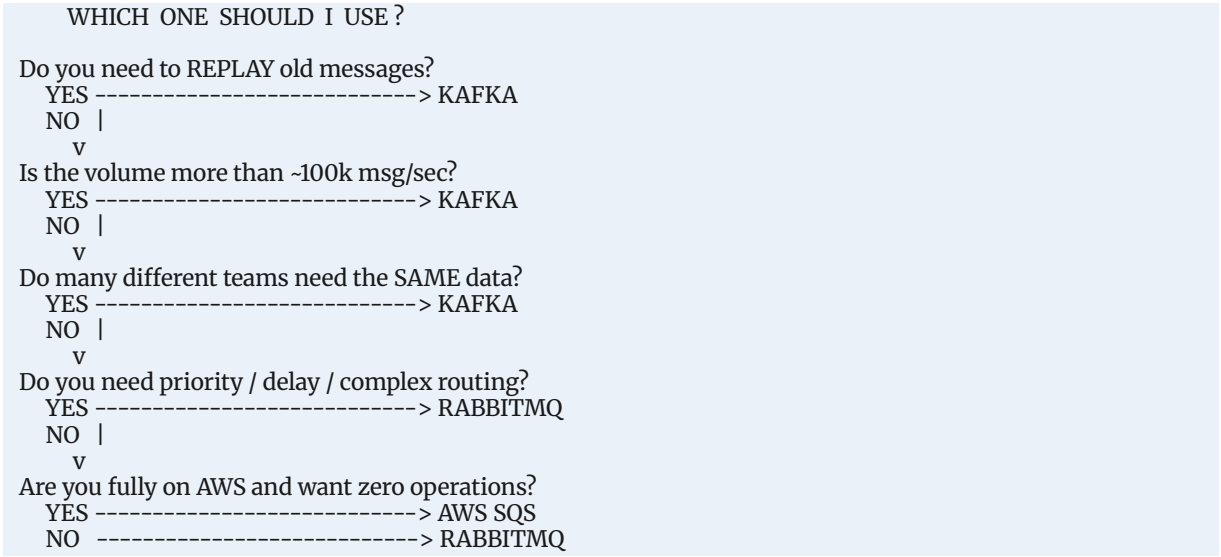


Figure 3.3 — Decision flow for choosing a messaging system

Practical note: These systems are not enemies. A typical large company uses RabbitMQ or SQS for small internal task queues such as "send this email", and Kafka for the high-volume event backbone that feeds analytics, machine learning and the data warehouse.

4. Kafka Architecture

Kafka uses a distributed client–server architecture. The servers are called brokers and form a cluster. The clients are producers and consumers. They talk to each other over a simple, efficient, language-independent binary protocol on top of TCP.

4.1 The Complete Architecture

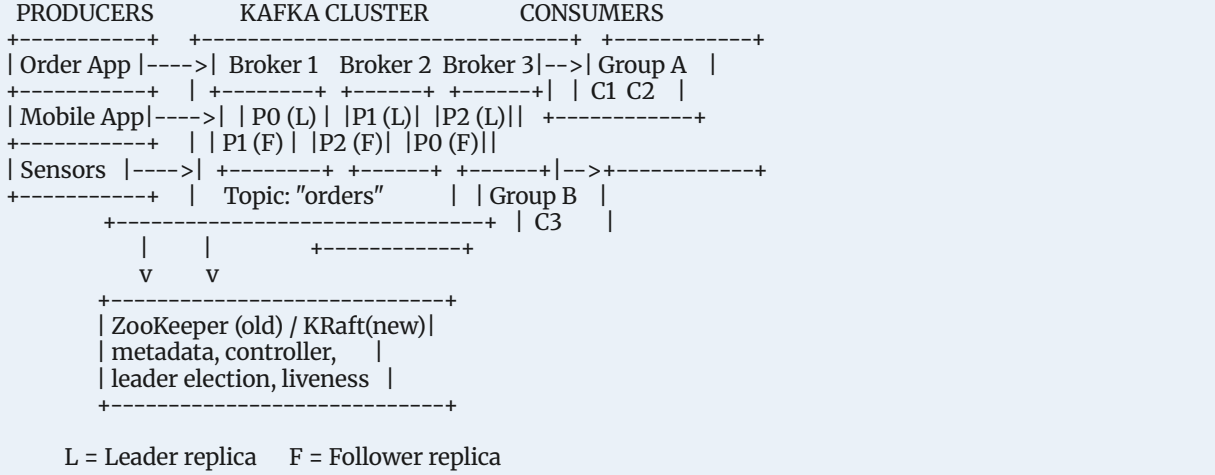


Figure 4.1 — Complete Kafka architecture

4.2 Every Component Explained

Component	Role	Key Point to Remember
Producer	Writes records into topics	Chooses the partition, batches and compresses
Consumer	Reads records from topics	Pulls data and tracks its own offset
Broker	A single Kafka server	Stores partitions as log files on disk
Cluster	A group of brokers	Shares partitions and replicas among brokers
Topic	A named stream of records	Logical grouping, like a table name
Partition	An ordered log inside a topic	The unit of parallelism and of ordering
Offset	Position of a record in a partition	Starts at 0, never reused, never changes
Replica	A copy of a partition	Gives durability and fault tolerance
Leader	The replica serving reads and writes	Exactly one leader per partition
Follower	A replica that copies the leader	Takes over if the leader dies
ISR	In-Sync Replicas	Replicas that are fully caught up
Controller	A special broker	Handles leader election and metadata

Component	Role	Key Point to Remember
ZooKeeper / KRaft	The coordination layer	KRaft is the modern replacement
Consumer Group	A team of consumers	Shares partitions; scales the reading side

4.3 How Data is Physically Stored

A partition is not one giant file. Kafka splits each partition into segment files so that old data can be deleted cheaply by simply removing whole files.

```
/kafka-logs/orders-0/        <-- topic "orders", partition 0
  00000000000000000000.log    <-- segment: actual messages
  00000000000000000000.index  <-- offset -> byte position
  00000000000000000000.timeindex <-- timestamp -> offset
  00000000000000123456.log    <-- next segment starts at 123456
  00000000000000123456.index
  leader-epoch-checkpoint

* The file name is the base offset of that segment.
* Only the newest segment is "active" and receives writes.
* Retention deletes whole old segment files - very cheap.
* The .index file lets Kafka jump directly to an offset
  instead of scanning the file from the start.
```

Figure 4.2 — Segment files on disk inside one partition

4.4 Replication, Leader and ISR

Replication is how Kafka survives hardware failure. Each partition is copied to several brokers. One copy is the leader; the rest are followers.

- All reads and writes go to the leader only. Followers never serve clients; they only copy data. This keeps consistency simple.
- Followers continuously fetch from the leader, just like a consumer would.
- A follower that is up to date is part of the ISR (In-Sync Replica set). A follower that falls behind by more than replica.lag.time.max.ms is removed from the ISR.
- If the leader dies, the controller picks a new leader from the ISR — never from an out-of-sync replica, because that would lose data.

```
Topic "orders", Partition 0, Replication Factor = 3

    +------------+     copy     +------------+
    | Broker 1  |  ------------>  | Broker 2  |
    | P0 LEADER |              | P0 follow |
    +------------+ \           +------------+
        ^       \  copy
        |        \        +------------+
write/read |          ------>  | Broker 3  |
        |               | P0 follow |
    [Producer/Consumer]        +------------+

ISR = { Broker1, Broker2, Broker3 }

Broker 1 crashes:
ISR = { Broker2, Broker3 }  ->  Broker 2 becomes LEADER
```

```
Clients automatically reconnect to the new leader.
RF = 3 tolerates the loss of 2 brokers.
```

Figure 4.3 — Replication and automatic leader election

High Watermark

A record is only visible to consumers after it has been copied to all in-sync replicas. The offset up to which all ISR members have the data is called the high watermark. This prevents a consumer from reading a record that could still be lost if the leader crashed immediately after writing it.

```
Leader log:   [0][1][2][3][4][5][6]
Follower A:   [0][1][2][3][4]
Follower B:   [0][1][2][3][4]
                    ^
            High Watermark = 4

Consumers can read offsets 0 to 4 only.
Offsets 5 and 6 exist but are not yet safe.
```

Figure 4.4 — The high watermark protects consumers from unsafe reads

4.5 ZooKeeper versus KRaft

For most of its life, Kafka needed a second system called ZooKeeper to store metadata and elect the controller. This meant running and monitoring two distributed systems. KRaft removes that dependency by letting Kafka manage its own metadata using the Raft consensus algorithm.

Point	ZooKeeper Mode (old)	KRaft Mode (new)
Metadata storage	Separate ZooKeeper ensemble	Internal Kafka topic @metadata
Processes to run	ZooKeeper cluster + Kafka brokers	Kafka only (controller and broker roles)
Setup complexity	Higher — two systems	Lower — one system
Partition limit	Around 200,000 partitions	Millions of partitions
Controller failover	Slow — must reload all metadata	Almost instant — metadata already in memory
Security	Two systems to secure separately	One security model
Status	Deprecated; removed in Kafka 4.0	Production-ready from Kafka 3.3

4.6 Why Kafka is Extremely Fast

Students are often asked this in exams. There are six main reasons.

1. Sequential disk writes. Kafka only appends to the end of a file. Sequential disk access is hundreds of times faster than random access and can even beat random access in RAM.

2. Zero-copy transfer. Using the sendfile() system call, data goes from the OS page cache straight to the network card without being copied into the application. This saves CPU and memory bandwidth.
3. Batching. The producer groups many small messages into one request, and the broker writes them together. Fewer, larger operations are far more efficient.
4. Compression. Whole batches are compressed with snappy, lz4 or zstd, cutting network and disk usage by 50–80 percent.
5. OS page cache. Kafka does not build its own cache. It relies on the operating system's file cache, so recent data is served from RAM with no extra work.
6. Partitioning. Work is split across many brokers, many disks and many consumers, all operating in parallel.

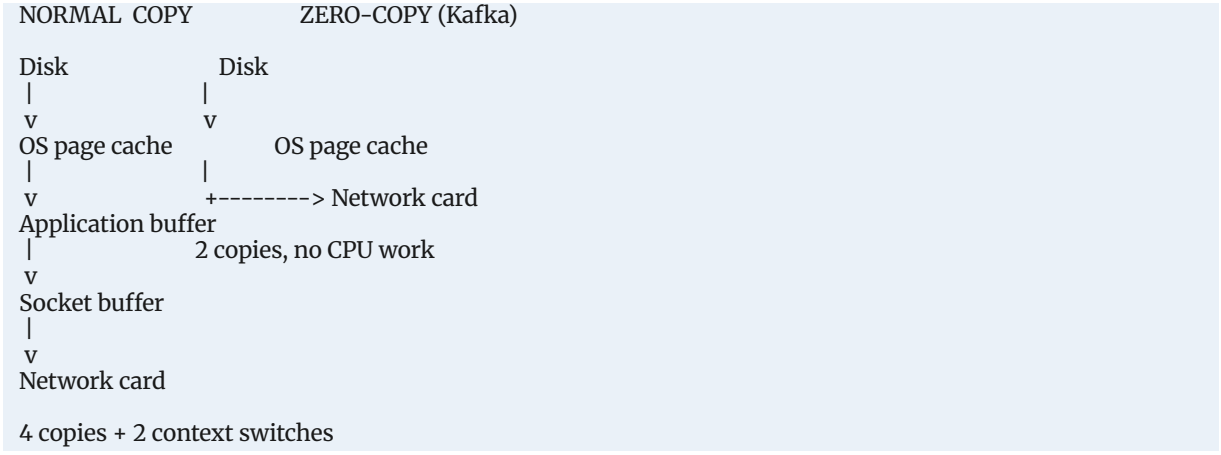


Figure 4.5 — Zero-copy avoids unnecessary data copying

5. Kafka Cluster and Broker

5.1 What is a Broker?

A broker is one Kafka server process running on one machine. Its job is simple but critical: accept writes, store them on disk, and serve reads.

- Unique identity: every broker has an integer ID set by broker.id (or node.id in KRaft). No two brokers in a cluster may share an ID.
- Stores partitions: a broker holds a subset of the partitions of many topics, as segment files under log.dirs.
- Stateless about consumers: the broker does not remember which consumer read which message. That is the consumer's job. This is why Kafka scales so well.
- Acts as a bootstrap server: a client connects to any one broker and immediately receives the full cluster metadata — the list of all brokers, topics, partitions and leaders.
- Handles replication: as a leader it serves followers; as a follower it fetches from other leaders.

5.2 What is a Cluster?

A cluster is a group of brokers that know about each other and share the work. Partitions and replicas are distributed so that no broker holds all the leaders and no two replicas of the same partition sit on the same machine.

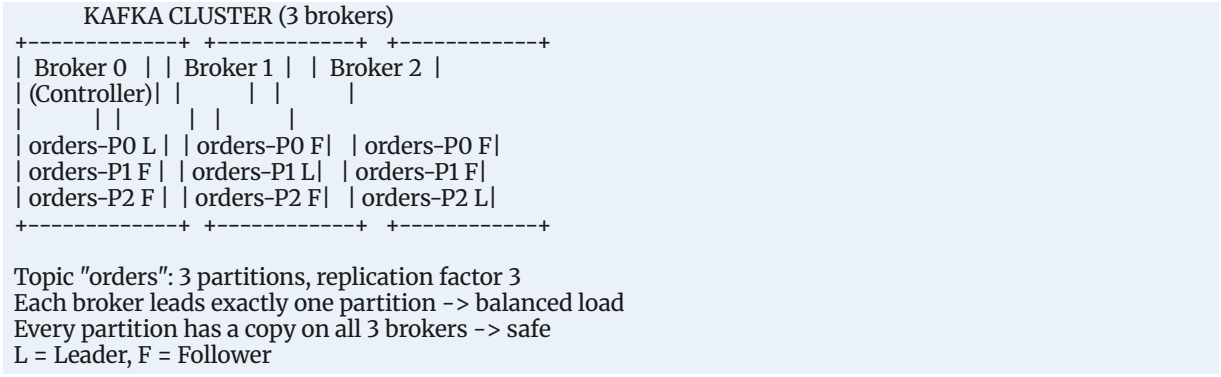


Figure 5.1 — Partitions and replicas spread across a 3-broker cluster

5.3 The Controller Broker

Exactly one broker in the cluster plays an additional role called the controller. It is elected automatically. Its responsibilities are:

- Watching which brokers are alive and which have failed.
- Electing a new leader for every partition that lost its leader.

- Handling topic creation, deletion and partition count changes.
- Propagating updated metadata to all other brokers.
- Managing partition reassignment when brokers are added or removed.

If the controller itself fails, the remaining brokers elect a new controller within seconds. The cluster never depends on one machine permanently.

5.4 What Happens When a Broker Fails — Step by Step

1. The broker stops sending heartbeats to the controller.
2. After zookeeper.session.timeout.ms (or the KRaft equivalent), the controller marks it as dead.
3. The controller finds every partition whose leader was on that broker.
4. For each such partition, it selects a new leader from the ISR.
5. The controller sends the updated metadata to all brokers.
6. Producers and consumers receive a NOT_LEADER error, refresh their metadata, and reconnect to the new leader automatically.
7. When the failed broker restarts, it rejoins as a follower and copies the data it missed until it is back in the ISR.

Total impact: a few seconds of retries inside the client libraries. Applications usually do not even notice.

5.5 Fault Tolerance and Replication Factor

A cluster with replication factor R survives the failure of (R - 1) brokers without losing data.

Replication Factor	Brokers That Can Fail	Data Stored	Used For
1	0	1 copy	Local development only — no safety at all
2	1	2 copies	Small clusters, limited safety
3	2	3 copies	Recommended standard for production
4 or more	3 or more	4+ copies	Very critical data; costs more disk and network

- Replication factor can never be greater than the number of brokers. With 2 brokers you cannot create a topic with RF = 3.
- min.insync.replicas works together with acks=all. If RF = 3 and min.insync.replicas = 2, a write succeeds only when at least 2 replicas have it. If

only 1 replica is alive, the producer gets an error instead of silently risking data loss.

- The classic safe combination is: RF = 3, min.insync.replicas = 2, acks = all. This tolerates one broker failure with no data loss and no downtime.

5.6 Important Broker Configuration Properties

Property	Meaning	Typical Value
broker.id	Unique ID of this broker	0, 1, 2
listeners	Address and port the broker binds to	PLAINTEXT://:9092
advertised.listeners	Address clients are told to use	PLAINTEXT://192.168.1.10:9092
log.dirs	Directories where partition data is stored	/data/kafka-logs
num.partitions	Default partitions for auto-created topics	3
log.retention.hours	How long messages are kept	168 (7 days)
log.segment.bytes	Size of one segment file	1073741824 (1 GB)
default.replication.factor	Default copies per partition	3
min.insync.replicas	Minimum ISR needed for acks=all	2
auto.create.topics.enable	Create a topic automatically on first use	false in production
num.network.threads	Threads handling network requests	3
num.io.threads	Threads handling disk I/O	8
message.max.bytes	Largest allowed message	1048576 (1 MB)

6. Topics and Partitions

6.1 Topic

A topic is a named category of records. It is a purely logical concept — like a table name in a database or a folder name in a file system. Producers write to a topic and consumers read from a topic.

- A topic can have any number of producers writing to it at the same time.
- A topic can have any number of consumer groups reading it, and each group gets all the data.
- Topic names should describe the data, not the consumer. Good: orders, payments, user-clicks. Bad: data-for-analytics-team.
- A common naming convention is domain.entity.action, for example shop.order.created.

6.2 Partition

A partition is the physical implementation of a topic. Each topic is split into one or more partitions, and each partition is an ordered, immutable, append-only sequence of records stored on disk.

Partitions exist for three reasons:

- Scalability: a topic can grow beyond the capacity of one machine, because different partitions live on different brokers.
- Parallelism: different consumers can read different partitions at the same time.
- Ordering control: related records can be forced into the same partition so their order is preserved.

```
          TOPIC : "orders"  (3 partitions)

Partition 0 :  [0][1][2][3][4][5]     <-- appended here
Partition 1 :  [0][1][2][3]          <--
Partition 2 :  [0][1][2][3][4][5][6]  <--
            ^
            |
          offset numbers, unique WITHIN each partition

* Order is guaranteed INSIDE a partition.
* Order is NOT guaranteed ACROSS partitions.
* Records are never modified - only appended.
* Each partition can live on a different broker.
```

Figure 6.1 — A topic divided into three partitions

6.3 The Ordering Rule — Explained With an Example

This is the most commonly misunderstood point in Kafka, and a very common exam question.

Wrong assumption: "Kafka keeps my messages in order."

Correct statement: "Kafka keeps messages in order within a single partition."

```
Producer sends 6 events in this order:
 E1 E2 E3 E4 E5 E6   (no key given -> round robin)

Partition 0 :  E1  E4
Partition 1 :  E2  E5
Partition 2 :  E3  E6

A consumer may read them as:  E1 E2 E3 E4 E5 E6
                  or:   E3 E6 E1 E4 E2 E5
                  or:   E2 E1 E5 E3 E4 E6

All are VALID. Global order is NOT guaranteed.
But E1 always comes before E4 (same partition).
```

Figure 6.2 — Ordering is guaranteed only inside a partition

6.4 How the Partition is Chosen

Situation	Rule Applied	Result
Partition number given explicitly	That exact partition is used	Full manual control
Key is present	partition = hash(key) % number_of_partitions	Same key always lands in the same partition
Key is null	Sticky partitioning (batch-wise round robin)	Even spread across all partitions
Custom partitioner class	Your own Java logic decides	Used for special business rules

Worked example: A topic "orders" has 3 partitions. A shopping app always sends customer events using the customer ID as the key.

```
send("orders", key="user_101", value="Added to cart")
    |  hash("user_101") % 3 = 1
    v
Partition 1   [ Added to cart ]

send("orders", key="user_101", value="Checked out")
    |  hash("user_101") % 3 = 1   (same key, same result)
    v
Partition 1   [ Added to cart ][ Checked out ]

send("orders", key="user_102", value="Added to cart")
    |  hash("user_102") % 3 = 0
    v
Partition 0   [ Added to cart ]

Result: user_101 events are always read in the correct
order. user_102 events are independent, so it does not
matter that they are in another partition.
```

Figure 6.3 — Key-based partitioning preserves per-key ordering

Warning about hot partitions: if 80 percent of your traffic uses the same key (for example a topic keyed by country where India dominates), one partition receives most of

the load while others stay idle. This is called a hot partition or data skew. The fix is to choose a key with higher cardinality, for example user ID instead of country.

6.5 Offsets

An offset is a simple sequential number identifying the position of a record inside a partition.

- Offsets start at 0 and increase by 1 for each new record.
- They are assigned by the broker at write time and never change.
- They are unique only within a partition, not within a topic.
- A record is uniquely identified by the triple: topic + partition + offset.
- Deleted records do not free their offsets; numbering continues forward.

Offset Term	Meaning
Log start offset	The oldest offset still available (older data was deleted by retention)
Log end offset	The offset that will be given to the next record written
Current offset	The position a consumer has read up to in memory
Committed offset	The position a consumer has saved, so it can resume after a restart
High watermark	The highest offset that is safely replicated and visible to consumers

6.6 Retention and Log Compaction

Kafka never deletes a record because it was read. Records are removed only by the retention policy. This is the property that makes replay possible.

Policy	Configuration	Behaviour	Use Case
Time-based deletion	retention.ms (default 7 days)	Deletes records older than the limit	Event streams, logs
Size-based deletion	retention.bytes	Deletes oldest records when the partition exceeds the size	Disk-limited systems
Log compaction	cleanup.policy=compact	Keeps only the latest value for each key	Current state, lookup tables
Infinite retention	retention.ms = -1	Never deletes anything	Event sourcing, audit logs

Log Compaction Explained With an Example

```
BEFORE compaction (topic "user-address", key = user id):

off: 0      1      2      3      4      5
   u1=Pune  u2=Delhi u1=Mumbai u3=Goa  u2=Agra  u1=Nashik

AFTER compaction:
```

```
  u3=Goa   u2=Agra  u1=Nashik

Only the LATEST value of each key is kept.
The topic now behaves like a table of current addresses.
A new consumer can rebuild the full state by reading it.
```

Figure 6.4 — Log compaction keeps only the newest value per key

Tombstone: to delete a key from a compacted topic, the producer sends a record with that key and a null value. This special record is called a tombstone and tells compaction to remove the key entirely.

6.7 Choosing the Number of Partitions

Rule	Explanation
More partitions = more parallelism	Maximum active consumers in a group equals the partition count
Partitions can be increased, never decreased	Decreasing would destroy ordering and lose data
Increasing breaks key affinity	hash(key) % N changes, so existing keys may move to a new partition
Each partition costs resources	Open file handles, memory, and longer leader election after failure
Leader election time grows	Thousands of partitions on a failed broker take longer to reassign
Practical starting point	Partitions = 2 to 3 times the expected number of consumers
Plan for growth	It is easier to start with 6 partitions than to add them later

A simple sizing method: if your target throughput is 100 MB/s and one partition can handle about 10 MB/s in your hardware, you need at least 10 partitions. Then add some headroom for future growth, so 15 to 20 would be a sensible choice.

7. Kafka Producer and Consumer

7.1 The Kafka Producer

A producer is any application that writes records into Kafka. The producer library does far more work than simply sending bytes: it serializes the data, decides the partition, groups records into batches, compresses them, sends them asynchronously, and retries automatically when something fails.

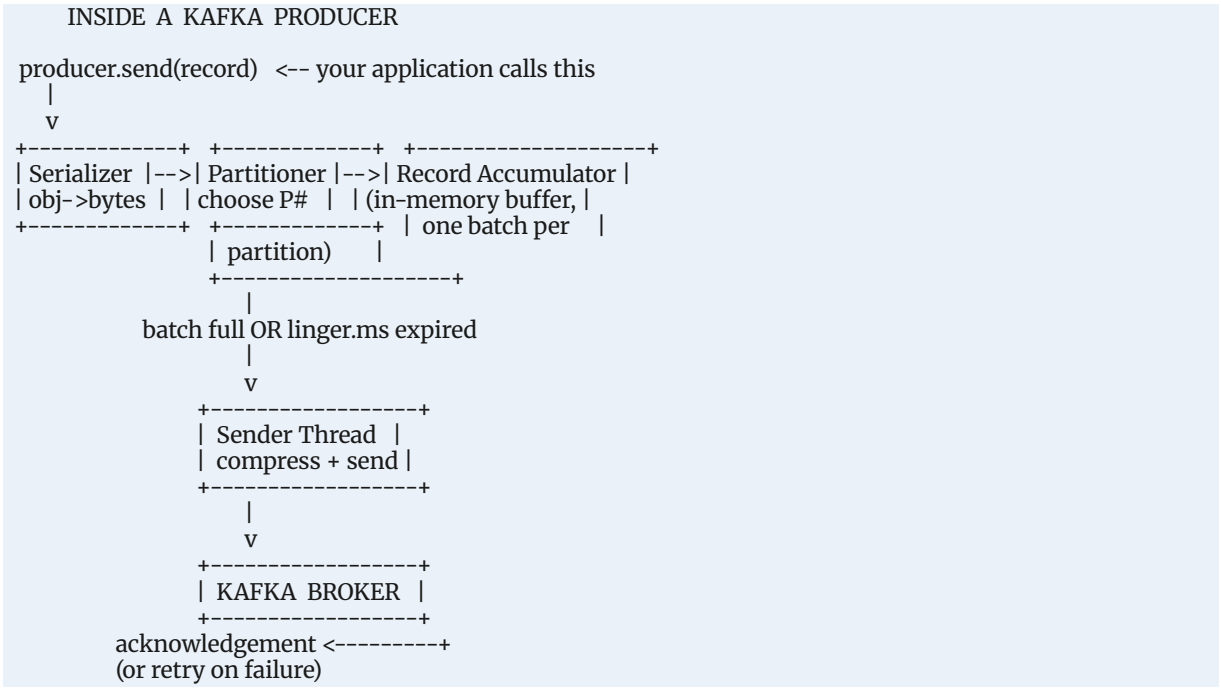


Figure 7.1 — Internal working of a Kafka producer

Step-by-Step What Happens on send()

1. Your application calls send() with a ProducerRecord. The call returns immediately — it is asynchronous.
2. The serializer converts the key and value objects into byte arrays.
3. The partitioner decides which partition the record belongs to.
4. The record is placed into an in-memory buffer called the record accumulator, in a batch for that partition.
5. A background sender thread waits until the batch reaches batch.size or until linger.ms has passed.
6. The batch is compressed and sent to the leader broker of that partition.
7. The broker writes it to disk and replies with an acknowledgement containing the partition and offset.
8. If the send fails with a retriable error, the producer retries automatically.

Important Producer Configurations

Configuration	Purpose	Default
bootstrap.servers	Initial list of brokers to contact	required
key.serializer / value.serializer	Converts objects into bytes	required
acks	How many replicas must confirm the write	1 (all in newer versions)
retries	How many times to retry a failed send	Integer.MAX_VALUE
batch.size	Maximum bytes per batch, per partition	16384 (16 KB)
linger.ms	How long to wait to fill a batch	0
buffer.memory	Total memory for buffering unsent records	33554432 (32 MB)
compression.type	none, gzip, snappy, lz4, zstd	none
enable.idempotence	Prevents duplicates caused by retries	true (Kafka 3.0+)
max.in.flight.requests.per.connection	Unacknowledged requests allowed at once	5
request.timeout.ms	How long to wait for a broker reply	30000
partitioner.class	Custom partitioning logic	default sticky partitioner

The acks Setting — The Most Important Producer Choice

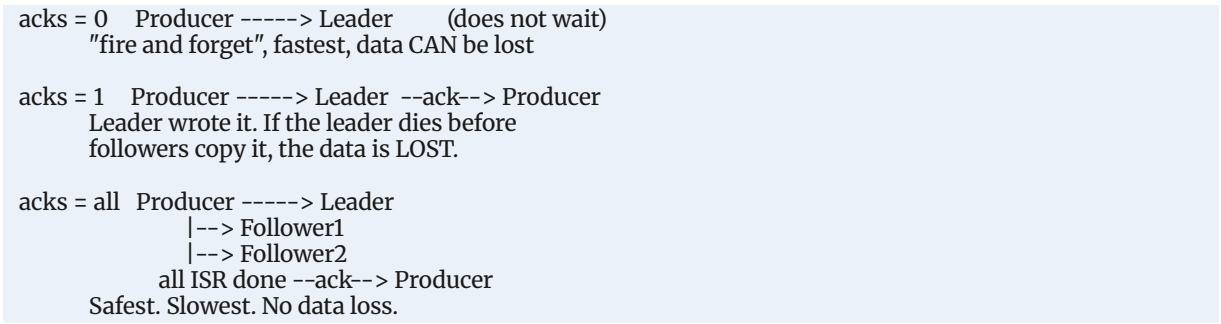


Figure 7.2 — The three acknowledgement levels

acks	Producer Waits For	Speed	Durability	When to Use
0	Nothing	Fastest	May lose data	Metrics, logs where loss is acceptable
1	Leader write only	Medium	Small risk of loss	General purpose, older default
all (-1)	Leader + all ISR	Slowest	No loss	Financial and critical data

Producer Example in Java

```
Properties props = new Properties();
props.put("bootstrap.servers", "localhost:9092,localhost:9093");
props.put("key.serializer",
  "org.apache.kafka.common.serialization.StringSerializer");
props.put("value.serializer",
  "org.apache.kafka.common.serialization.StringSerializer");
props.put("acks", "all");          // strongest durability
props.put("enable.idempotence", "true");  // no duplicates
props.put("compression.type", "snappy");
props.put("linger.ms", "10");      // wait 10ms to batch

KafkaProducer<String, String> producer = new KafkaProducer<>(props);

ProducerRecord<String, String> record =
  new ProducerRecord<>("orders", "user_101", "Laptop,45000");

// asynchronous send with a callback
producer.send(record, (metadata, ex) -> {
  if (ex == null) {
    System.out.println("topic="    + metadata.topic()
                + " partition=" + metadata.partition()
                + " offset="   + metadata.offset());
  } else {
    ex.printStackTrace();
  }
});

producer.flush();   // force all buffered records to be sent
producer.close();
```

Producer Example in Python

```
from kafka import KafkaProducer
import json

producer = KafkaProducer(
  bootstrap_servers=["localhost:9092"],
  key_serializer=lambda k: k.encode("utf-8"),
  value_serializer=lambda v: json.dumps(v).encode("utf-8"),
  acks="all",
  retries=5,
  compression_type="snappy")

for i in range(10):
  producer.send("orders",
          key=f"user_{i}",
          value={"item": "Laptop", "amount": 45000})

producer.flush()
producer.close()
```

7.2 The Kafka Consumer

A consumer is an application that reads records from one or more topics. Kafka deliberately uses a pull model: the consumer repeatedly asks the broker for more data rather than the broker pushing data to the consumer.

Why Pull Instead of Push?

Reason	Explanation
No overload	A slow consumer simply asks for less; a fast producer cannot flood it

Reason	Explanation
Efficient batching	The consumer can fetch many records in one request
Consumer controls the speed	It can pause, slow down, or catch up whenever it wants
Replay is natural	The consumer just asks for an older offset
Simpler broker	The broker does not track consumer state at all

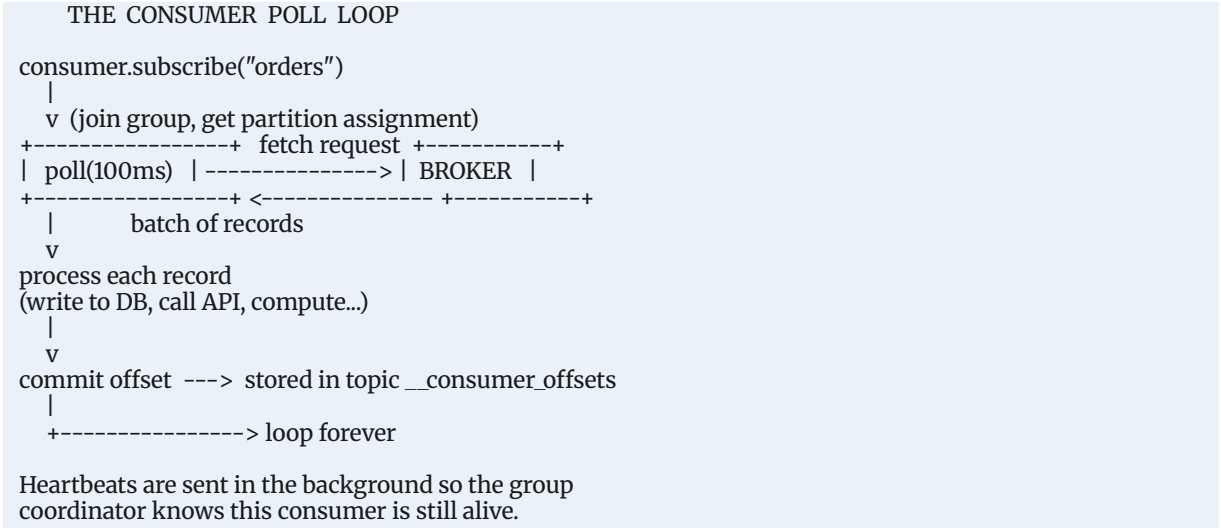
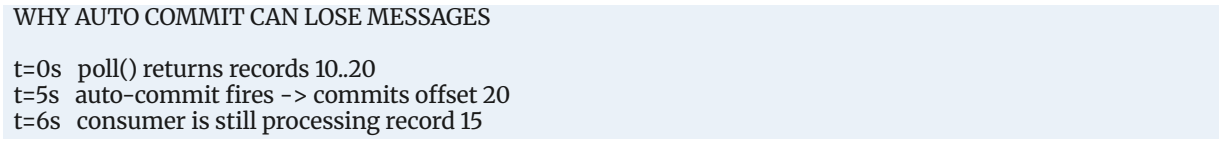


Figure 7.3 — The consumer poll loop

Offset Management

The committed offset is stored in a special internal Kafka topic called __consumer_offsets. Because it is stored in Kafka itself, a consumer can crash, restart on a different machine, and still continue exactly where it stopped.

Method	How It Works	Advantage	Risk
Auto commit	enable.auto.commit=true, commits in the background every 5 s	Simplest, no code needed	Can lose or duplicate messages after a crash
Manual sync	commitSync() after processing; blocks until confirmed	Safe and predictable	Slower, blocks the loop
Manual async	commitAsync() with a callback; does not block	Fast	A failed commit may go unnoticed
Per-record commit	Commit after every single record	Maximum precision	Very slow, heavy load on the broker



```
t=7s   consumer CRASHES

On restart it resumes from offset 20.
Records 15..20 were never processed -> LOST.

Fix: enable.auto.commit=false and call commitSync()
    only AFTER processing is complete.
```

Figure 7.4 — The danger of auto-commit

auto.offset.reset decides where a consumer starts when it has no committed offset (a brand-new group, or the old offset has been deleted by retention): earliest reads from the very beginning of the partition, latest reads only records arriving from now on, and none throws an exception.

Consumer Example in Java

```
Properties props = new Properties();
props.put("bootstrap.servers", "localhost:9092");
props.put("group.id", "order-processing-group");
props.put("key.deserializer",
  "org.apache.kafka.common.serialization.StringDeserializer");
props.put("value.deserializer",
  "org.apache.kafka.common.serialization.StringDeserializer");
props.put("enable.auto.commit", "false");   // manual control
props.put("auto.offset.reset", "earliest");
props.put("max.poll.records", "100");

KafkaConsumer<String, String> consumer = new KafkaConsumer<>(props);
consumer.subscribe(Arrays.asList("orders"));

try {
  while (true) {
    ConsumerRecords<String, String> records =
        consumer.poll(Duration.ofMillis(100));

    for (ConsumerRecord<String, String> r : records) {
      System.out.printf("p=%d off=%d key=%s value=%s%n",
          r.partition(), r.offset(), r.key(), r.value());
      // ... business logic here ...
    }
    consumer.commitSync();   // commit AFTER processing
  }
} finally {
  consumer.close();   // leaves the group cleanly
}
```

Consumer Example in Python

```
from kafka import KafkaConsumer
import json

consumer = KafkaConsumer(
  "orders",
  bootstrap_servers=["localhost:9092"],
  group_id="order-processing-group",
  enable_auto_commit=False,
  auto_offset_reset="earliest",
  value_deserializer=lambda v: json.loads(v.decode("utf-8")))

for message in consumer:
  print(message.partition, message.offset, message.value)
  consumer.commit()
```

7.3 Message Delivery Semantics

This decides what happens when something goes wrong. It is a very frequent exam question.

Semantic	Meaning	How It Happens	Use Case
At most once	Message may be lost, never duplicated	Commit the offset BEFORE processing	Metrics where losing a point is fine
At least once	Message never lost, may be duplicated	Commit the offset AFTER processing (default)	Most applications
Exactly once	Delivered once and only once	Idempotent producer + transactions + read_committed	Payments, billing, inventory

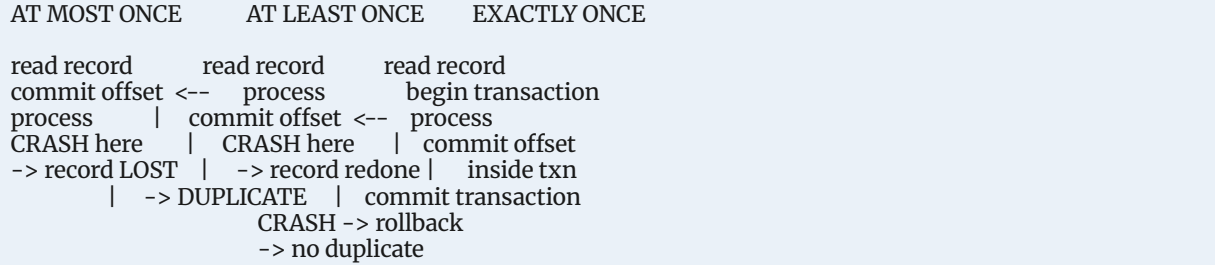


Figure 7.5 — Where the crash happens decides the semantic

Idempotent Producer

When enable.idempotence is true, the producer attaches a producer ID and a sequence number to every record. If a retry sends the same record twice, the broker sees the repeated sequence number and silently ignores the duplicate. This gives exactly-once guarantees per partition without any extra work from the developer.

Handling Duplicates in Practice

Because at-least-once is the common default, real applications make their processing idempotent — meaning that processing the same message twice has the same effect as processing it once.

- Use the Kafka offset or a unique business ID as the primary key in the database, so a repeat insert simply fails or overwrites.
- Use "UPSERT" (insert or update) instead of plain insert.
- Keep a small table of processed message IDs and skip anything already seen.

8. Message Serialization: Avro, Protobuf and Schema Registry

8.1 What is Serialization and Why is it Needed?

Kafka brokers understand only one thing: an array of bytes. They have no idea whether your message is a Java object, a Python dictionary or a JSON document. Therefore every message must be converted.

- Serialization: the producer converts an object into a byte array before sending.
- Deserialization: the consumer converts the byte array back into an object after receiving.

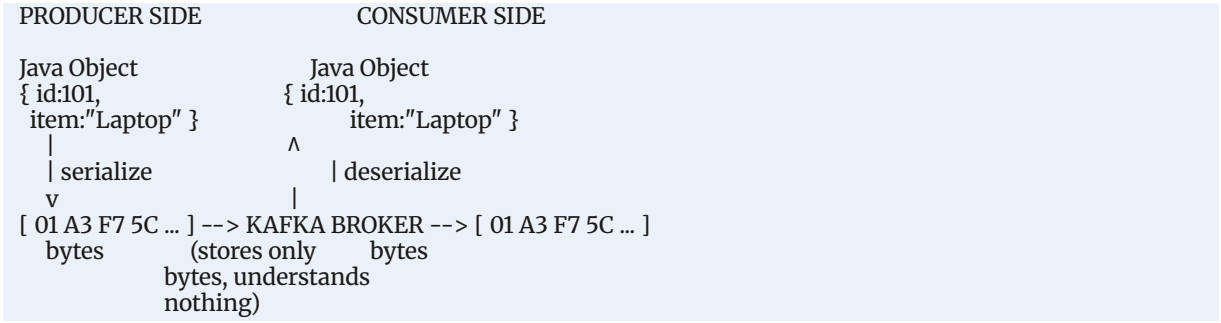


Figure 8.1 — Serialization and deserialization around Kafka

Kafka ships with built-in serializers for simple types: StringSerializer, IntegerSerializer, LongSerializer, DoubleSerializer, ByteArraySerializer. For real business objects you need a proper format.

8.2 Why Plain JSON is a Bad Idea at Scale

JSON is convenient and human-readable, so beginners always start with it. At scale it causes four serious problems.

Problem	Explanation	Impact
Repeated field names	Every message repeats "orderId", "customer", "amount"	Messages are 3–10 times bigger than needed
No schema enforcement	A producer can rename or remove a field any time	Consumers break silently in production
No type safety	A field that was a number may arrive as a string	Runtime crashes and bad data
Slow parsing	Text must be parsed character by character	Higher CPU usage on every consumer

```
THE BREAKAGE STORY

Monday  : producer sends { "amount": 45000 }
```

```
        10 consumers read "amount" happily

Tuesday  : a developer renames it to "amt"
        producer tests pass, code is deployed

Tuesday  : 10 consumers crash at 2 AM
 night     because "amount" no longer exists

Nobody was warned. Nothing checked it.
A Schema Registry would have REJECTED this change.
```

Figure 8.2 — What happens without schema enforcement

8.3 Apache Avro

Avro is a compact binary serialization format from the Apache Hadoop ecosystem. It is the most widely used format in Kafka pipelines.

Key Characteristics

- Schema written separately in JSON (a .avsc file) describing every field and its type.
- Data stored in pure binary, without field names, so a message that is 120 bytes in JSON may be only 30 bytes in Avro.
- Excellent schema evolution — fields can be added with default values so old consumers keep working.
- Rich type system — records, enums, arrays, maps, unions, nullable fields, and logical types such as date, decimal and timestamp.
- No code generation required (though it is supported), because the schema can be read at runtime.

Example Avro Schema — Order.avsc

```
{
 "type": "record",
 "name": "Order",
 "namespace": "com.shop.events",
 "doc": "An order placed by a customer",
 "fields": [
  { "name": "orderId",  "type": "string" },
  { "name": "customer", "type": "string" },
  { "name": "item",     "type": "string" },
  { "name": "amount",   "type": "double" },
  { "name": "status",   "type": {
     "type": "enum", "name": "Status",
     "symbols": ["PLACED","SHIPPED","DELIVERED"] },
    "default": "PLACED" },
  { "name": "city",     "type": ["null","string"],
    "default": null }
 ]
}
```

Notice two things: the field "city" has the union type ["null","string"] with a default of null, which makes it optional; and "status" is an enum with a default. Both patterns allow the schema to be extended later without breaking any existing consumer.

8.4 Protocol Buffers (Protobuf)

Protobuf is a binary serialization format created by Google. The schema is written in a .proto file, and a compiler (protoc) generates strongly-typed classes for Java, Python, Go, C++, C# and many other languages.

Key Characteristics

- Fields are identified by numbers, not names. Renaming a field is completely safe as long as the number stays the same.
- Extremely compact and fast — usually slightly smaller and faster than Avro.
- Requires code generation, which gives compile-time type safety in your application.
- Natural choice with gRPC, so teams already using gRPC microservices often use Protobuf in Kafka too.
- Never reuse a field number. Deleted numbers should be marked "reserved" so no one reuses them by mistake.

Example Protobuf Schema — order.proto

```
syntax = "proto3";
package com.shop.events;

enum Status {
  PLACED    = 0;
  SHIPPED   = 1;
  DELIVERED = 2;
}

message Order {
  string orderId  = 1;
  string customer = 2;
  string item     = 3;
  double amount   = 4;
  Status status   = 5;
  string city     = 6;  // added later - safe, new tag

  reserved 7, 8;        // never reuse these numbers
}
```

8.5 Comparison of Serialization Formats

Aspect	JSON	Avro	Protobuf
Encoding	Text	Binary	Binary
Typical message size	Large (100%)	Small (~30%)	Smallest (~25%)
Serialization speed	Slow	Fast	Fastest
Schema needed	No	Yes (.avsc, JSON)	Yes (.proto)

Aspect	JSON	Avro	Protobuf
Schema sent with data	Not applicable	Only a 4-byte schema ID	Only a schema ID
Human readable	Yes	No	No
Code generation	Not needed	Optional	Required
Field identity	By name	By name and position	By number
Schema evolution	Not managed at all	Excellent, via defaults	Excellent, via field numbers
Language support	Universal	Very good	Excellent
Best for	Prototypes, debugging	Kafka and big-data pipelines	Microservices and gRPC

```
SIZE COMPARISON OF THE SAME RECORD

JSON     : {"orderId":"A1","customer":"user_101",
           "item":"Laptop","amount":45000.0}
         = about 78 bytes

Avro     : [magic][schemaID][A1|user_101|Laptop|45000]
         = about 28 bytes   (field names removed)

Protobuf : [magic][schemaID][1:A1][2:user_101]...
         = about 26 bytes

Over 1 crore messages per day this saves tens of GB
of network traffic and disk space.
```

Figure 8.3 — Why binary formats matter at scale

8.6 Schema Registry

The Schema Registry is a separate lightweight service (most commonly the Confluent Schema Registry) that stores every schema centrally, assigns each one a unique numeric ID, keeps version history, and enforces compatibility rules.

How It Works — Step by Step

1. The first time a producer uses a schema, it sends that schema to the registry. The registry stores it and returns a schema ID (for example 42).
2. The producer writes to Kafka only a magic byte, the 4-byte schema ID, and the binary data — never the full schema.
3. The consumer reads the message and extracts the schema ID from the first bytes.
4. The consumer asks the registry for schema 42, then caches it locally. It will never ask again for that ID.
5. The consumer deserializes the binary data using that schema.

6. If a producer later tries to register an incompatible schema, the registry rejects the request and the deployment fails immediately — before any consumer breaks.

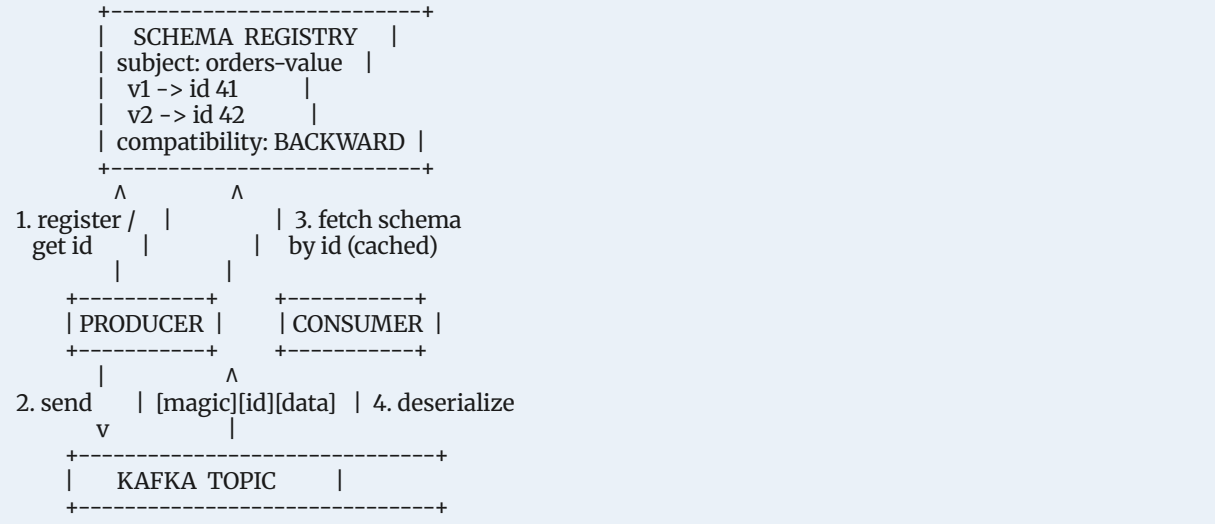


Figure 8.4 — Working of the Schema Registry

The Wire Format

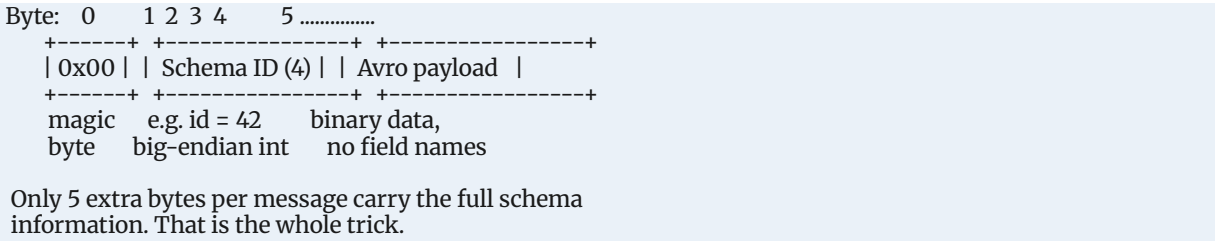


Figure 8.5 — Confluent wire format: magic byte + schema ID + payload

Subjects and Versions

A subject is the name under which schemas are registered. By default it is topicName-key and topicName-value. Each subject keeps a version history, so you can see exactly how the data structure of a topic changed over time.

Compatibility Types

Type	Guarantee	Allowed Changes	Upgrade Order
BACKWARD (default)	New schema can read old data	Delete a field, add an optional field	Consumers first
FORWARD	Old schema can read new data	Add a field, delete an optional field	Producers first
FULL	Both directions work	Add or delete only optional fields	Any order
NONE	No checking at all	Anything — dangerous	Not recommended

Type	Guarantee	Allowed Changes	Upgrade Order
*_TRANSITIVE	Checked against ALL past versions, not just the last	Same as above but stricter	Safest

Benefits of Using a Schema Registry

- It is the single source of truth for the structure of every topic in the company.
- It blocks incompatible changes before they reach production, turning a 2 AM outage into a failed build.
- It reduces message size dramatically because the schema is never repeated.
- Producers and consumers can be upgraded independently and at different times.
- It automatically provides versioned documentation of all the data in the organisation.
- New team members can discover what data exists by browsing the registry.

Configuration With Avro

```
// ---------- Producer ----------
props.put("key.serializer",
  "io.confluent.kafka.serializers.KafkaAvroSerializer");
props.put("value.serializer",
  "io.confluent.kafka.serializers.KafkaAvroSerializer");
props.put("schema.registry.url", "http://localhost:8081");

// ---------- Consumer ----------
props.put("key.deserializer",
  "io.confluent.kafka.serializers.KafkaAvroDeserializer");
props.put("value.deserializer",
  "io.confluent.kafka.serializers.KafkaAvroDeserializer");
props.put("schema.registry.url", "http://localhost:8081");
props.put("specific.avro.reader", "true");
```

Useful Schema Registry REST Commands

```
# List all subjects
curl http://localhost:8081/subjects

# List the versions of one subject
curl http://localhost:8081/subjects/orders-value/versions

# Fetch a specific version
curl http://localhost:8081/subjects/orders-value/versions/1

# Check the compatibility setting
curl http://localhost:8081/config/orders-value

# Change the compatibility setting
curl -X PUT -H "Content-Type: application/json" \
    --data '{"compatibility":"FULL"}' \
   http://localhost:8081/config/orders-value
```

9. Consumer Groups

9.1 What is a Consumer Group?

A consumer group is a set of consumer instances that share the same group.id and cooperate to read a topic. Kafka divides the partitions of the topic among the members, so each member processes only a part of the data. This is the mechanism by which Kafka scales the reading side.

A consumer group gives you two things at once:

- Load balancing inside the group: partitions are shared, so adding consumers increases throughput.
- Publish–subscribe across groups: every group receives a complete, independent copy of all the data.

9.2 The Golden Rules

Rule 1: Within one group, each partition is assigned to exactly one consumer. Two consumers in the same group never read the same partition.

Rule 2: One consumer may be assigned several partitions.

Rule 3: If the number of consumers exceeds the number of partitions, the extra consumers remain idle and do nothing.

Rule 4: Therefore the maximum useful parallelism of a group equals the number of partitions.

Rule 5: Different consumer groups are completely independent — each keeps its own offsets and receives all the messages.

9.3 Group Sizes — Three Cases

```
Topic "orders" with 4 partitions (P0 P1 P2 P3)

CASE 1: Group A has 2 consumers  (under-provisioned)
  P0, P1  ----->  Consumer 1
  P2, P3  ----->  Consumer 2      balanced, 2 each

CASE 2: Group A has 4 consumers  (ideal)
  P0 ----> C1     P1 ----> C2
  P2 ----> C3     P3 ----> C4     perfect, 1 each

CASE 3: Group A has 5 consumers  (over-provisioned)
  P0 ----> C1     P1 ----> C2
  P2 ----> C3     P3 ----> C4
             C5 ----> IDLE   wasted resource

To use 5 consumers you must first increase the topic
to at least 5 partitions.
```

Figure 9.1 — Partition assignment for different group sizes

9.4 Two Groups Reading the Same Topic

This is how Kafka achieves publish–subscribe. Each team creates its own consumer group and reads the same topic without affecting anyone else.

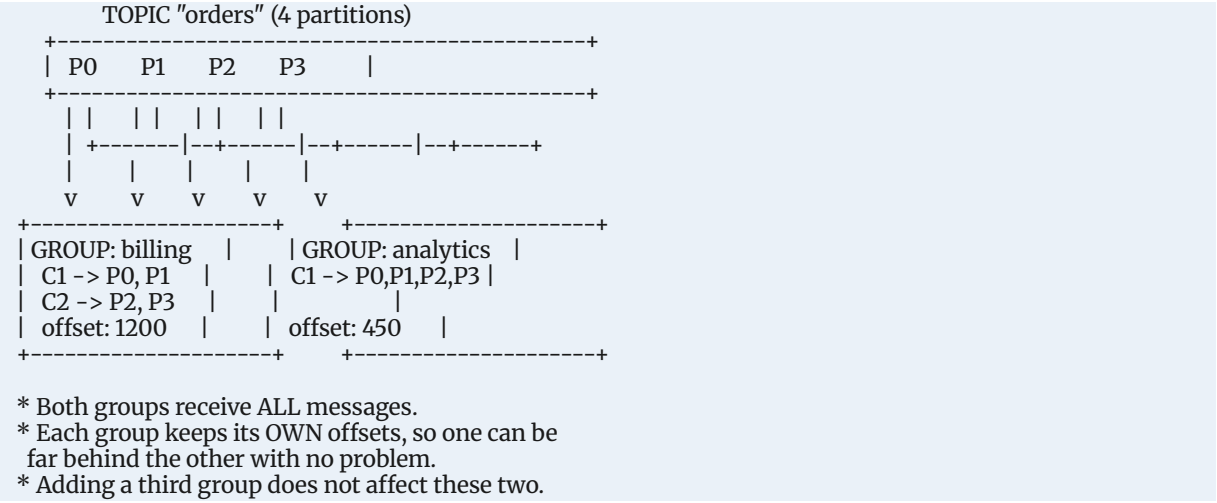


Figure 9.2 — Publish–subscribe through multiple consumer groups

9.5 Group Coordinator and Group Leader

Role	Who Plays It	Responsibilities
Group Coordinator	One broker in the cluster	Tracks membership through heartbeats, stores committed offsets, triggers rebalances
Group Leader	The first consumer that joined the group	Runs the assignment strategy and decides which partition goes to which consumer

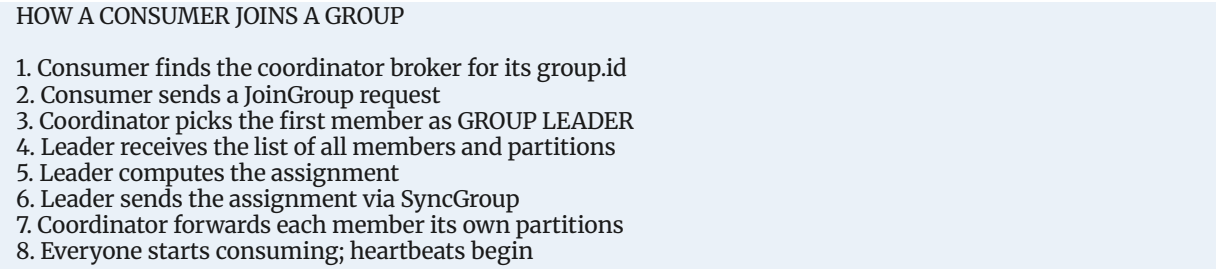


Figure 9.3 — The group join protocol

9.6 Rebalancing

Rebalancing is the process of redistributing partitions among the members of a group. It is what makes Kafka consumers self-healing and elastic.

When Does a Rebalance Happen?

- A new consumer joins the group (scaling up).
- A consumer leaves the group gracefully by calling close().

- A consumer crashes and stops sending heartbeats within session.timeout.ms.
- A consumer takes longer than max.poll.interval.ms between two poll() calls, so it is considered stuck.
- The number of partitions in the topic is increased.
- A subscribed topic matching a pattern is created or deleted.

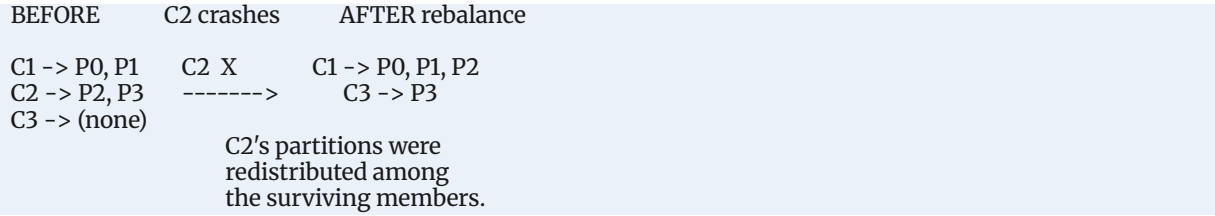


Figure 9.4 — Rebalancing after a consumer failure

Eager versus Cooperative Rebalancing

Aspect	Eager (old)	Cooperative (modern)
Behaviour	All consumers give up all partitions, then get new ones	Only the partitions that must move are revoked
Pause	Whole group stops consuming ("stop the world")	Most consumers keep working
Rounds	One round	Two small rounds
Impact on a large group	Can pause for several seconds	Almost no visible impact
Enabled by	RangeAssignor, RoundRobinAssignor	CooperativeStickyAssignor

Practical advice: always set partition.assignment.strategy to CooperativeStickyAssignor in new applications, and always call consumer.close() on shutdown so the group rebalances immediately instead of waiting for the session timeout.

9.7 Partition Assignment Strategies

Strategy	How It Works	Problem It Has
RangeAssignor	For each topic, assigns continuous ranges of partitions	Uneven when consuming several topics
RoundRobinAssignor	Distributes all partitions one by one across consumers	Assignment changes completely on rebalance
StickyAssignor	Balanced and tries to preserve previous assignments	Still uses eager rebalancing
CooperativeStickyAssignor	Sticky plus incremental rebalancing	Recommended; needs newer clients

9.8 Important Consumer Group Settings

Setting	Meaning	Default
group.id	Name of the consumer group	required
session.timeout.ms	Time without heartbeats after which a consumer is declared dead	45000
heartbeat.interval.ms	How often heartbeats are sent (about one third of the session timeout)	3000
max.poll.interval.ms	Maximum allowed gap between two poll() calls	300000
max.poll.records	Maximum records returned by one poll()	500
fetch.min.bytes	Broker waits until this much data is available	1
fetch.max.wait.ms	How long the broker waits to collect fetch.min.bytes	500
partition.assignment.strategy	Which assignment strategy to use	Range, CooperativeSticky

9.9 Consumer Lag — The Key Metric

Consumer Lag = Log End Offset - Committed Offset

It tells you how many messages a consumer group still has to process. It is the single most important number to monitor in any Kafka system.

```
Partition 0
[0][1][2][3][4][5][6][7][8][9][10][11][12]
            ^                 ^
            |                 |
      committed offset = 5    log end offset = 13

      LAG = 13 - 5 = 8 messages behind

Lag steady and small   -> healthy
Lag steadily growing   -> consumers too slow: add more
                          consumers or more partitions
Lag suddenly huge      -> a consumer has crashed
```

Figure 9.5 — Understanding consumer lag

9.10 Scaling a Consumer Group — Worked Example

A topic "payments" has 6 partitions and receives 6000 messages per second. One consumer can process 1500 messages per second.

Consumers	Partitions Each	Capacity	Result
1	6	1500 msg/s	Lag grows fast — badly under-provisioned
2	3	3000 msg/s	Lag still grows

Consumers	Partitions Each	Capacity	Result
4	1 or 2	6000 msg/s	Exactly keeping up — no headroom
6	1	9000 msg/s	Healthy, with headroom for spikes
8	1 (2 idle)	9000 msg/s	No benefit — 2 consumers sit idle

Conclusion: six consumers is the correct answer here. To go beyond that the topic must first be given more partitions.

10. Single-Node and Multi-Broker Kafka Setup

10.1 Prerequisites

- Java 8 or above installed; Java 11 or Java 17 is recommended.
- Kafka binary downloaded from kafka.apache.org and extracted.
- At least 2 GB of free RAM for a small test setup.
- On Windows, use the scripts inside bin\windows\ with the .bat extension instead of .sh.

```
# Download and extract
wget https://downloads.apache.org/kafka/3.7.0/kafka_2.13-3.7.0.tgz
tar -xzf kafka_2.13-3.7.0.tgz
cd kafka_2.13-3.7.0

# Verify Java
java -version

# What is inside
ls
#  bin/     -> all the command line tools
#  config/   -> server.properties, zookeeper.properties etc.
#  libs/     -> jar files
#  logs/     -> broker application logs
```

10.2 Single-Node Single-Broker Setup

This is the simplest setup, used for learning and local development. Everything runs on one machine and the replication factor must be 1, so there is no fault tolerance.

```
    SINGLE NODE, SINGLE BROKER
 +--------------------------------------+
 |         One Machine            |
 |                          |
 |  ZooKeeper   : port 2181          |
 |       |                    |
 |       v                    |
 |  Broker 0    : port 9092          |
 |  log.dirs = /tmp/kafka-logs        |
 |                          |
 |  Topic "test": 1 partition, RF = 1  |
 +--------------------------------------+

 Good for learning. NOT fault tolerant.
```

Figure 10.1 — Single-node single-broker setup

Step-by-Step Procedure

```
# STEP 1 - Start ZooKeeper (Terminal 1)
bin/zookeeper-server-start.sh config/zookeeper.properties

# STEP 2 - Start the Kafka broker (Terminal 2)
bin/kafka-server-start.sh config/server.properties

# STEP 3 - Create a topic (Terminal 3)
bin/kafka-topics.sh --create --topic test \
   --bootstrap-server localhost:9092 \
   --partitions 1 --replication-factor 1
```

```
# STEP 4 - Verify the topic was created
bin/kafka-topics.sh --list --bootstrap-server localhost:9092

# STEP 5 - Start a console producer and type some messages
bin/kafka-console-producer.sh --topic test \
    --bootstrap-server localhost:9092
> hello kafka
> this is my first message

# STEP 6 - In Terminal 4, start a console consumer
bin/kafka-console-consumer.sh --topic test --from-beginning \
    --bootstrap-server localhost:9092
```

Important Lines in config/server.properties

```
broker.id=0
listeners=PLAINTEXT://:9092
log.dirs=/tmp/kafka-logs
num.partitions=1
zookeeper.connect=localhost:2181
log.retention.hours=168
offsets.topic.replication.factor=1
```

10.3 Single-Node Multi-Broker Setup

Here three brokers run on the same machine. This lets you practise replication, leader election and fault tolerance without needing three computers. Each broker must differ in three properties, otherwise they will clash on ports and data directories.

```
    SINGLE NODE, MULTI BROKER (3 brokers)
+-----------------------------------------------------+
|              One Machine                |
|                                  |
|   ZooKeeper : 2181                      |
|     |        |        |              |
|     v        v        v              |
| Broker 0    Broker 1   Broker 2            |
|  :9092       :9093      :9094             |
| logs-0       logs-1     logs-2            |
+-----------------------------------------------------+

Now replication factor 3 is possible.
Killing one broker demonstrates leader election.
```

Figure 10.2 — Single-node multi-broker setup

Configuration Differences

File	broker.id	listeners	log.dirs
server.properties	0	PLAINTEXT://:9092	/tmp/kafka-logs-0
server-1.properties	1	PLAINTEXT://:9093	/tmp/kafka-logs-1
server-2.properties	2	PLAINTEXT://:9094	/tmp/kafka-logs-2

```
# STEP 1 - Copy the config file twice
cp config/server.properties config/server-1.properties
cp config/server.properties config/server-2.properties

# STEP 2 - Edit the three properties in each copy
```

```
#   server-1.properties: broker.id=1, port 9093, logs-1
#   server-2.properties: broker.id=2, port 9094, logs-2

# STEP 3 - Start ZooKeeper and then all three brokers
bin/zookeeper-server-start.sh config/zookeeper.properties &
bin/kafka-server-start.sh config/server.properties   &
bin/kafka-server-start.sh config/server-1.properties &
bin/kafka-server-start.sh config/server-2.properties &

# STEP 4 - Create a replicated topic
bin/kafka-topics.sh --create --topic multi-topic \
    --bootstrap-server localhost:9092 \
    --partitions 3 --replication-factor 3

# STEP 5 - Inspect leaders, replicas and ISR
bin/kafka-topics.sh --describe --topic multi-topic \
    --bootstrap-server localhost:9092
```

Expected Output of --describe

```
Topic: multi-topic  PartitionCount: 3  ReplicationFactor: 3
 Topic: multi-topic Partition: 0 Leader: 0 Replicas: 0,1,2 Isr: 0,1,2
 Topic: multi-topic Partition: 1 Leader: 1 Replicas: 1,2,0 Isr: 1,2,0
 Topic: multi-topic Partition: 2 Leader: 2 Replicas: 2,0,1 Isr: 2,0,1
```

Testing Fault Tolerance

```
# Kill broker 1
ps aux | grep server-1.properties     # find the PID
kill -9 <PID>

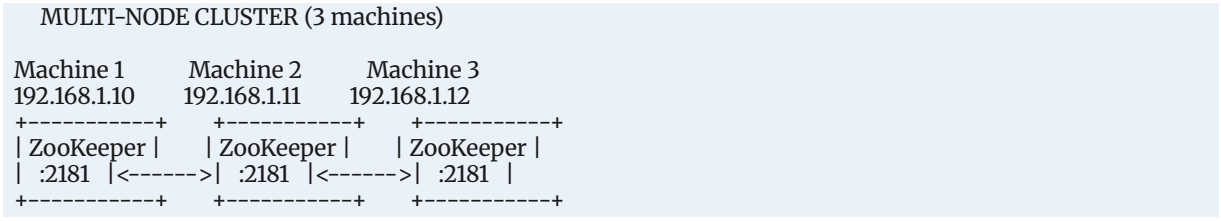
# Describe the topic again
bin/kafka-topics.sh --describe --topic multi-topic \
    --bootstrap-server localhost:9092

# Observe:
#  - Partition 1 now has a different Leader
#  - Broker 1 has disappeared from the Isr list
#  - Replicas still shows 1,2,0 (the assignment did not change)
#  - Producers and consumers keep working with NO data loss
```

10.4 Multi-Node (True Distributed) Cluster

In production each broker runs on a separate machine. The procedure is the same, with two crucial additions.

- advertised.listeners must be the hostname or IP that clients can actually reach. If it is left as localhost, external clients will receive "localhost" as the broker address and fail to connect. This is the single most common setup mistake.
- All brokers must point to the same ZooKeeper ensemble (or the same KRaft controller quorum). If they point to different ones, you get separate clusters instead of one.



```
| Broker 1 |      | Broker 2 |      | Broker 3 |
|  :9092  |<------>|  :9092  |<------>|  :9092  |
+-----------+      +-----------+      +-----------+

Note: ZooKeeper needs an ODD number of nodes (3, 5, 7)
so it can always form a majority.
```

Figure 10.3 — A three-machine distributed cluster

```
# ---------- On machine 1 (192.168.1.10) ----------
broker.id=1
listeners=PLAINTEXT://0.0.0.0:9092
advertised.listeners=PLAINTEXT://192.168.1.10:9092
log.dirs=/data/kafka-logs
zookeeper.connect=192.168.1.10:2181,192.168.1.11:2181,\
            192.168.1.12:2181
default.replication.factor=3
min.insync.replicas=2

# On machine 2 -> broker.id=2, advertised IP 192.168.1.11
# On machine 3 -> broker.id=3, advertised IP 192.168.1.12
```

10.5 KRaft Mode Setup (No ZooKeeper)

KRaft is the modern way to run Kafka. There is no ZooKeeper at all — Kafka stores its own metadata using the Raft consensus protocol.

```
# STEP 1 - Generate a unique cluster ID (only once, ever)
KAFKA_CLUSTER_ID="$(bin/kafka-storage.sh random-uuid)"
echo $KAFKA_CLUSTER_ID

# STEP 2 - Format the storage directory with that ID
bin/kafka-storage.sh format -t $KAFKA_CLUSTER_ID \
    -c config/kraft/server.properties

# STEP 3 - Start Kafka (no ZooKeeper needed)
bin/kafka-server-start.sh config/kraft/server.properties

# Key properties in config/kraft/server.properties
#   process.roles=broker,controller
#   node.id=1
#   controller.quorum.voters=1@localhost:9093
#   listeners=PLAINTEXT://:9092,CONTROLLER://:9093
```

process.roles Value	Meaning	Used In
broker	This node only stores data and serves clients	Large production clusters
controller	This node only manages metadata	Large production clusters
broker,controller	This node does both	Development and small clusters

10.6 Running Kafka With Docker (Quick Alternative)

```
# docker-compose.yml (KRaft, single broker)
version: "3"
services:
  kafka:
    image: bitnami/kafka:latest
```

```
  ports:
    - "9092:9092"
  environment:
    - KAFKA_CFG_NODE_ID=0
    - KAFKA_CFG_PROCESS_ROLES=controller,broker
    - KAFKA_CFG_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093
    - KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=0@kafka:9093

# Start it
docker compose up -d
```

10.7 Common Setup Mistakes

Mistake	Symptom	Fix
advertised.listeners left as localhost	External clients cannot connect	Set it to the real IP or hostname
Same log.dirs for two brokers	Broker fails to start, lock error	Give each broker its own directory
Duplicate broker.id	Second broker refuses to start	Use unique IDs
RF greater than broker count	Topic creation fails	RF must be ≤ number of brokers
ZooKeeper not started first	Broker cannot connect and exits	Always start ZooKeeper before brokers
Firewall blocking 9092	Connection timeout from clients	Open the port on all broker machines

11. Basic Kafka Commands and Operations

Note: On Linux and macOS use the .sh scripts inside bin/. On Windows use the same names with .bat inside bin\windows\. In modern Kafka always use --bootstrap-server; the old --zookeeper option has been removed completely.

11.1 Topic Commands

```
# Create a topic
bin/kafka-topics.sh --create --topic orders \
    --bootstrap-server localhost:9092 \
    --partitions 3 --replication-factor 2

# Create with extra configuration
bin/kafka-topics.sh --create --topic orders \
    --bootstrap-server localhost:9092 \
    --partitions 3 --replication-factor 2 \
    --config retention.ms=604800000 \
    --config cleanup.policy=delete

# List all topics
bin/kafka-topics.sh --list --bootstrap-server localhost:9092

# Describe one topic (partitions, leader, replicas, ISR)
bin/kafka-topics.sh --describe --topic orders \
    --bootstrap-server localhost:9092

# Describe all topics at once
bin/kafka-topics.sh --describe --bootstrap-server localhost:9092

# Find unhealthy partitions
bin/kafka-topics.sh --describe --under-replicated-partitions \
    --bootstrap-server localhost:9092

# Increase the number of partitions (never decrease)
bin/kafka-topics.sh --alter --topic orders --partitions 6 \
    --bootstrap-server localhost:9092

# Delete a topic
bin/kafka-topics.sh --delete --topic orders \
    --bootstrap-server localhost:9092
```

11.2 Producer Commands

```
# Simple console producer - type and press Enter
bin/kafka-console-producer.sh --topic orders \
    --bootstrap-server localhost:9092

# Producer with keys; input format ->  user_101:Laptop
bin/kafka-console-producer.sh --topic orders \
    --bootstrap-server localhost:9092 \
    --property parse.key=true --property key.separator=:

# Send the contents of a file
bin/kafka-console-producer.sh --topic orders \
    --bootstrap-server localhost:9092 < data.txt

# Producer with acks=all
bin/kafka-console-producer.sh --topic orders \
    --bootstrap-server localhost:9092 \
    --producer-property acks=all
```

11.3 Consumer Commands

```
# Read only NEW messages from now on
bin/kafka-console-consumer.sh --topic orders \
    --bootstrap-server localhost:9092

# Read ALL messages from the very beginning
bin/kafka-console-consumer.sh --topic orders --from-beginning \
    --bootstrap-server localhost:9092

# Read as part of a consumer group, showing keys and partitions
bin/kafka-console-consumer.sh --topic orders --group grp1 \
    --property print.key=true \
    --property print.partition=true \
    --property print.offset=true \
    --property print.timestamp=true \
    --bootstrap-server localhost:9092

# Read from one specific partition, starting at offset 5
bin/kafka-console-consumer.sh --topic orders \
    --partition 0 --offset 5 \
    --bootstrap-server localhost:9092

# Read only 10 messages and then stop
bin/kafka-console-consumer.sh --topic orders --from-beginning \
    --max-messages 10 --bootstrap-server localhost:9092

# Look at the internal offsets topic
bin/kafka-console-consumer.sh --topic __consumer_offsets \
    --formatter "kafka.coordinator.group.\
GroupMetadataManager\$OffsetsMessageFormatter" \
    --bootstrap-server localhost:9092
```

11.4 Consumer Group Commands

```
# List all consumer groups
bin/kafka-consumer-groups.sh --list \
    --bootstrap-server localhost:9092

# Describe a group: offsets, log-end-offset and LAG
bin/kafka-consumer-groups.sh --describe --group grp1 \
    --bootstrap-server localhost:9092

# Show which member holds which partition
bin/kafka-consumer-groups.sh --describe --group grp1 --members \
    --bootstrap-server localhost:9092

# Reset offsets to the beginning (group must be STOPPED)
bin/kafka-consumer-groups.sh --reset-offsets --to-earliest \
    --group grp1 --topic orders --execute \
    --bootstrap-server localhost:9092

# Other reset options
#   --to-latest        jump to the end
#   --to-offset 50     jump to a specific offset
#   --shift-by -100    move back 100 messages
#   --to-datetime 2026-09-18T00:00:00.000
#   --dry-run          preview without applying

# Delete a consumer group
bin/kafka-consumer-groups.sh --delete --group grp1 \
    --bootstrap-server localhost:9092
```

Reading the --describe Output

```
GROUP  TOPIC   PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
grp1   orders  0          120             150             30
grp1   orders  1          200             200             0
grp1   orders  2          90              145             55

CURRENT-OFFSET  = where the group has committed up to
LOG-END-OFFSET  = the newest offset in that partition
LAG             = LOG-END-OFFSET - CURRENT-OFFSET

Total lag here = 30 + 0 + 55 = 85 messages behind.
Partition 2 is the slowest - check that consumer.
```

11.5 Configuration Commands

```
# Change retention of a topic to 1 day
bin/kafka-configs.sh --alter --entity-type topics \
    --entity-name orders --add-config retention.ms=86400000 \
    --bootstrap-server localhost:9092

# View all configs of a topic
bin/kafka-configs.sh --describe --entity-type topics \
    --entity-name orders --bootstrap-server localhost:9092

# Remove a config override (back to the broker default)
bin/kafka-configs.sh --alter --entity-type topics \
    --entity-name orders --delete-config retention.ms \
    --bootstrap-server localhost:9092

# View broker-level configuration
bin/kafka-configs.sh --describe --entity-type brokers \
    --entity-name 0 --bootstrap-server localhost:9092
```

11.6 Testing and Maintenance Commands

```
# Producer performance test
bin/kafka-producer-perf-test.sh --topic orders \
    --num-records 100000 --record-size 100 --throughput -1 \
    --producer-props bootstrap.servers=localhost:9092

# Consumer performance test
bin/kafka-consumer-perf-test.sh --topic orders \
    --messages 100000 --bootstrap-server localhost:9092

# Find the earliest and latest offsets of a topic
bin/kafka-get-offsets.sh --bootstrap-server localhost:9092 \
    --topic orders

# Reassign partitions after adding a new broker
bin/kafka-reassign-partitions.sh \
    --bootstrap-server localhost:9092 \
    --reassignment-json-file reassign.json --execute

# Trigger a preferred-leader election (rebalance leadership)
bin/kafka-leader-election.sh --bootstrap-server localhost:9092 \
    --election-type PREFERRED --all-topic-partitions

# Delete records before a given offset
bin/kafka-delete-records.sh --bootstrap-server localhost:9092 \
    --offset-json-file delete.json

# Stop a broker gracefully
bin/kafka-server-stop.sh
```

11.7 Quick Command Reference

Task	Script	Key Option
Create, list, describe, delete topics	kafka-topics.sh	--create --list --describe --delete
Send test messages	kafka-console-producer.sh	--topic
Read test messages	kafka-console-consumer.sh	--from-beginning --group
Manage groups and offsets	kafka-consumer-groups.sh	--describe --reset-offsets
Change topic or broker config	kafka-configs.sh	--alter --add-config
Start a broker	kafka-server-start.sh	config/server.properties
Stop a broker	kafka-server-stop.sh	—
Start ZooKeeper	zookeeper-server-start.sh	config/zookeeper.properties
Format storage for KRaft	kafka-storage.sh	format -t <id>
Performance testing	kafka-producer-perf-test.sh	--num-records
Move partitions between brokers	kafka-reassign-partitions.sh	--execute
Rebalance partition leadership	kafka-leader-election.sh	--election-type PREFERRED

12. End-to-End Case Study: A Food Delivery App

This section connects every concept in the unit into one realistic example, which is very useful for answering long-form exam questions.

12.1 The Requirement

A food delivery company needs to handle 50,000 orders per hour. When a customer places an order, five different things must happen: the restaurant must be notified, payment must be processed, a delivery partner must be assigned, the customer must receive an SMS, and the analytics dashboard must be updated.

12.2 The Design

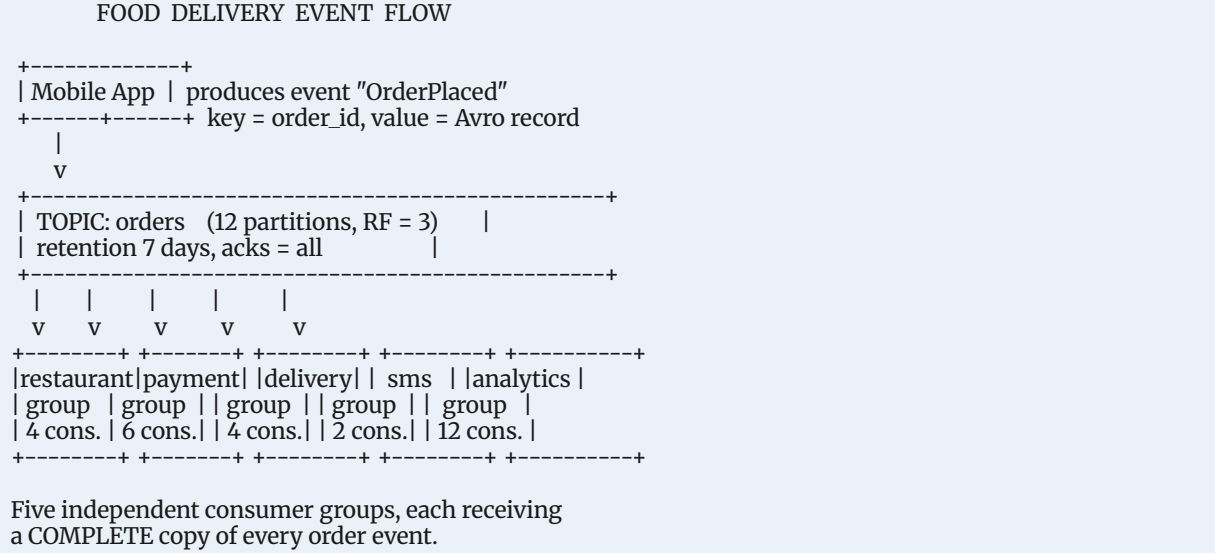


Figure 12.1 — One topic, five independent consumer groups

12.3 Design Decisions Explained

Decision	Choice Made	Reason
Partition count	12	Allows up to 12 parallel consumers per group and room to grow
Partition key	order_id	All events of one order stay in order; high cardinality avoids hot partitions
Replication factor	3	Survives the loss of 2 brokers
min.insync.replicas	2	With acks=all, guarantees no data loss on one broker failure
acks	all	Orders are money — losing one is unacceptable
Serialization	Avro + Schema Registry	Small messages and no silent breakage when fields change

Decision	Choice Made	Reason
Retention	7 days	Enough to replay a week if a consumer has a bug
Consumer commits	Manual, after processing	Gives at-least-once; duplicates handled by idempotent writes

12.4 What Happens When Things Go Wrong

Failure	What Kafka Does	Customer Impact
A broker crashes	Leaders move to other brokers from the ISR within seconds	None
The SMS service goes down for 2 hours	Events accumulate in the topic; lag grows	SMS arrives late, everything else is fine
The analytics team deploys a bug	They reset their offset and reprocess 3 days of data	None — other groups are unaffected
Traffic doubles during a festival	Add consumers to each group up to 12	None, if partitions are sufficient
A developer renames a field	Schema Registry rejects the change at deploy time	None — caught before production

13. Summary and Quick Revision

13.1 Glossary

Term	One-Line Definition
Event / Record	A single message with key, value, timestamp and headers
Producer	Application that writes records into a topic
Consumer	Application that reads records from a topic
Broker	A single Kafka server that stores partitions
Cluster	A group of brokers working as one system
Topic	A named logical stream of records
Partition	An ordered append-only log inside a topic; unit of parallelism
Offset	The position number of a record inside a partition
Segment	A physical file that stores part of a partition
Replication Factor	The number of copies kept of each partition
Leader	The replica that serves all reads and writes
Follower	A replica that only copies the leader
ISR	In-Sync Replicas — replicas fully caught up with the leader
High Watermark	The highest offset that is safely replicated and readable
Controller	The broker that manages leader election and metadata
Consumer Group	A set of consumers sharing the partitions of a topic
Group Coordinator	The broker managing a consumer group
Rebalance	Reassignment of partitions when group membership changes
Consumer Lag	Log-end offset minus committed offset
Retention	How long records are kept before deletion
Log Compaction	Keeping only the latest value for each key
Tombstone	A record with a null value that deletes a key in a compacted topic
Schema Registry	Central service that stores and versions message schemas
Idempotent Producer	A producer that cannot create duplicates when retrying
KRaft	Kafka's own consensus protocol, replacing ZooKeeper